

University of Tripoli

Faculty of Engineering

Department of Computer Engineering

Deep Reinforcement Learning for Adaptive Traffic Engineering in Software-Defined Networks with High-Availability Control

A Graduation Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Engineering

Prepared by:

Maher Abdulnasir Alqadhi

Student ID: 2210249576

Supervised by:

Dr. Suad El-Geder

Associate Professor

Academic Year 2025–2026

Certificate of Approval

This is to certify that the graduation project entitled:

"Deep Reinforcement Learning for Adaptive Traffic Engineering in Software-Defined Networks with High-Availability Control"

submitted by **Maher Abdulnasir Alqadhi** (Student ID: **2210249576**) in partial fulfillment of the requirements for the degree of **Bachelor of Science in Computer Engineering** has been approved by the examining committee after public defense and examination.

Supervisor:
Dr. Suad El-Geder, Associate Professor Signature: . . . . . .

Internal Examiner:
Prof./Dr. . . . . Signature: . . . . . .

External Examiner:
Prof./Dr. . . . . Signature: . . . . . .

Head of Department:
Department of Computer Engineering, University of Tripoli Signature: . . . . . .

Date of Defense: June 15, 2026

Declaration of Academic Integrity

I, the undersigned **Maher Abdulnasir Alqadhi**, hereby declare that this graduation project report entitled **"Deep Reinforcement Learning for Adaptive Traffic Engineering in Software-Defined Networks with High-Availability Control"** is the outcome of my own original research and engineering work conducted under the academic supervision of **Dr. Suad El-Geder** at the Department of Computer Engineering, Faculty of Engineering, University of Tripoli.

I affirm that:

1. All conceptual designs, mathematical derivations, software algorithms, neural network implementations, simulation harnesses, and experimental evaluation suites presented in this document were developed by me, except where explicit bibliographic reference and citation are made to published academic literature.

2. No portion of this work has been submitted previously for any academic degree, diploma, or professional qualification at this university or any other institution of higher learning.

3. All sources of technical literature, software libraries, and data sets utilized in the realization of this project have been fully acknowledged in the bibliography in accordance with academic standards.

Maher Abdulnasir Alqadhi
Tripoli, Libya
June 2026

. . . . . .
Candidate Signature

Dedication

To my beloved parents, whose unconditional love, sacrifices, prayers, and steadfast belief in my potential have been the guiding light throughout my life and academic journey;

To my brothers and sisters, for their unwavering encouragement, patience, and companionship;

To all my esteemed professors and teachers at the University of Tripoli, who generously shared their knowledge, wisdom, and passion for engineering excellence;

And to every passionate researcher and student striving to harness technology and artificial intelligence for the advancement of society.

Acknowledgement

First and foremost, all praise, gratitude, and adoration are due to Allah Almighty, the Most Merciful and Most Gracious, for granting me the perseverance, health, intellectual clarity, and strength to complete this graduation dissertation and reach this major academic milestone.

I wish to express my deepest, most sincere gratitude and utmost respect to my academic supervisor, **Dr. Suad El-Geder**, for her exceptional mentorship, invaluable guidance, continuous encouragement, and rigorous academic critique throughout the conception, development, and evaluation of this research. Her technical expertise in telecommunications and computer networking, along with her unyielding insistence on scientific precision, provided the cornerstone for overcoming critical engineering obstacles and improving the rigor of this work.

I also extend my sincere appreciation to the distinguished faculty members and technical staff of the **Department of Computer Engineering, Faculty of Engineering at the University of Tripoli**. I am deeply indebted for the comprehensive foundational education, rigorous laboratory training, and inspiring scholarly environment they provided throughout my undergraduate curriculum.

No words can adequately convey my profound debt of gratitude to my beloved parents; to my father for his lifelong sacrifices, wisdom, and steadfast belief in my potential, and to my mother for her unconditional love, endless prayers, and constant moral support. My heartfelt appreciation also goes to my brothers and sisters for their continuous encouragement and patience during the demanding phases of this project.

Finally, I warmly thank my fellow colleagues, classmates, and friends at the University of Tripoli who accompanied me along this demanding academic path, sharing fruitful technical debates, collaborative problem solving, and steadfast camaraderie.

Maher Abdulnasir Alqadhi

Abstract

Modern telecommunication infrastructures and cloud datacenter fabrics are constrained by non-stationary traffic dynamics, localized microbursts, and rigid routing heuristics. Traditional interior gateway protocols—predominantly Open Shortest Path First (OSPF, RFC 2328) and Equal-Cost Multi-Path (ECMP)—rely on static administrative metrics decoupled from instantaneous buffer occupancy. Under asymmetric traffic loads, these protocols persistently direct traffic along identical shortest paths, inducing core link saturation, bufferbloat, tail-drop packet loss, and packet delay variation (jitter), while viable edge-disjoint bypass paths remain underutilized.

This graduation dissertation presents the design, implementation, and evaluation of a closed-loop Adaptive Traffic Engineering (TE) prototype powered by Deep Reinforcement Learning (DRL) over an OpenFlow 1.3 Software-Defined Networking (SDN) substrate. The control plane integrates a Dueling Double Deep Q-Network (D3QN), a binary SumTree Prioritized Experience Replay (PER) engine, and a thresholded multi-objective reward. The agent consumes normalized 10-dimensional link and candidate-path features and selects among loopless candidate paths synthesized with Yen's K-shortest-paths algorithm, with invalid actions masked. A health-checked active/standby controller pool provides automated failover resilience, delivering an internal local role promotion bookkeeping time of 0.001 ms and an end-to-end client HTTP recovery of 37.96 ms under simulated primary process failure.

The evaluation benchmarks D3QN across five network topologies against both static baselines (OSPF, Dijkstra SPF, ECMP) and dynamic heuristics (Least-Loaded Routing, LLR, and Widest-Shortest Path, WSP). In the multi-topology tournament under heavy load, D3QN achieves substantial congestion relief across multipath fabrics, lowering arithmetic-mean bottleneck utilization from 66.80% (OSPF) to 30.80% and reducing modelled latency from 158.34 ms to 44.50 ms. However, the evaluation also demarcates clear operational boundaries: on rigid tree topologies with single root bottlenecks, greedy dynamic heuristics like LLR deliver comparable or superior throughput (38.09 Mbps vs. 34.08 Mbps for D3QN) without neural inference overhead. Furthermore, empirical 120-trial Mininet namespace emulations reveal that dynamic multi-path switching interacts with TCP window congestion control, resulting in higher packet loss under TCP than static SPF (36.0% vs. 18.0%). Consequently, this work establishes that DRL-based traffic engineering is most advantageous on rich multipath fabrics with diverse edge-disjoint routing options, while deterministic heuristics remain competitive in single-bottleneck architectures.

Contents

Certificate of Approval ii

Declaration of Academic Integrity iii

Dedication iv

Acknowledgement v

Abstract vi

List of Figures xi

List of Tables xiii

List of Abbreviations xiv

1 Introduction 1

1.1 Problem Statement 1
1.2 The Importance of the Experiment and Why It is Important 2
1.3 Solution Methodology and Main Steps 2
1.4 Project Contributions 3
1.5 Scope, Assumptions, and Operational Boundaries 4
1.6 Organization of the Dissertation 5

2 Background 7

2.1 Hardware, Software Tools, and Key Equipment: Why Each Was Chosen . . 7
2.2 Programmable Data Planes (P4) and In-Band Network Telemetry (INT) . . 9
2.3 Mathematical Foundations of Traffic Engineering and UMCF 10
2.4 Queueing Delay, Bufferbloat, and RFC 3393 Jitter Modeling 12
2.5 Deep Reinforcement Learning Foundations 13
2.6 Dueling Double Deep Q-Networks (D3QN) 14
2.7 Prioritized Experience Replay (PER) with Binary SumTree 15

3 Related Work 16

3.1 Overview and Taxonomy of Traffic Engineering 16
3.2 Classical Heuristic and Static Metric Optimization 16

3.3 Centralized SDN Heuristics and Linear Solvers 16
3.4 Deep Reinforcement Learning in Network Routing 17
3.5 High Availability and Resilience in SDN Control Planes 18
3.6 Systematic Comparison of Related Work 18
3.7 Research Gap and Key Innovations 18

4 The Proposed System **20**
4.1 Design 20
4.1.1 Top-Level Closed-Loop Control Architecture 20
4.1.2 Continuous 10-Dimensional State Space Representation and Action Masking 21
4.1.3 Candidate Action Space Pruning via Yen's K-Shortest Paths 22
4.1.4 D3QN Neural Network Architecture 23
4.1.5 Implemented Multi-Objective Reward Function 24
4.1.6 Binary SumTree Prioritized Experience Replay Architecture 24
4.1.7 Health-Checked Active/Standby Multi-Controller Pool 25
4.1.8 Closed-Loop Control Execution Timeline 26
4.2 Experiments 26
4.2.1 Evaluation Environment and Testbed Provisioning 26
4.2.2 Benchmark Network Topologies 27
4.2.3 Synthetic, Bursty, and Adversarial Traffic Generation 28
4.2.4 Baseline Routing Protocols for Comparative Tournament 28
4.3 Testing and Evaluation 29
4.3.1 Key Performance Metrics 29
4.3.2 Automated Test Suite and Functional Verification 29
4.3.3 Claim-to-Evidence Verification Matrix 30
4.4 Results and Discussion 30
4.4.1 Comprehensive Multi-Topology Routing Tournament Results . . . 30
4.4.2 Latency, Jitter, and Packet Loss Distributions 32
4.4.3 Empirical Packet-Level Mininet Emulation (120-Trial Factorial Evaluation) 32
4.4.4 Traffic Load Scaling and Congestion Transition 35
4.4.5 Multi-Seed Statistical ANOVA and Variance Decomposition 36
4.4.6 Ablation Studies: Dissecting Policy Guards and Architectural Components 38
4.4.7 Extreme Stress Testing: Jamming and Burst Avalanches 39
4.4.8 Zero-Shot Generalization on Unseen Random Topologies 41
4.4.9 Comparison Against Theoretical Optimal HiGHS MILP Solver . . 41
4.4.10 High-Availability Controller Failover Benchmarks 43

4.4.11 Critical Discussion: Operational Trade-offs and Computational Costs 44
4.4.12 Threats to Validity and Mitigation Strategies 45

5 Conclusion **46**
5.1 Restatement of the Problem 46
5.2 Summary of Proposed System and Methodology 46
5.3 Quantitative Impact and Performance Milestones 47
5.4 Restrictions, Engineering Limits, and Weaknesses 47

6 Future Work **49**
6.1 P4 Programmable Data Planes and In-Band Network Telemetry 49
6.2 Multi-Agent RL for Inter-Domain Carrier Transit 49
6.3 Graph Neural Networks for Topological Invariance 49
6.4 Constrained MDPs and Formal Safety Verification 50
6.5 Optical Transport Network and Reconfigurable Wavelength Routing 50
6.6 Zero-Trust Control-Plane Cryptographic Attestation 50

References **51**

A Mathematical Formulations and Proofs **55**
A.1 Identifiability of the Dueling Architecture 55

B OpenFlow 1.3 Flow Table and REST API Specifications **56**
B.1 OpenFlow 1.3 Flow Table Entry Structure 56
B.2 REST API Endpoints 56

C Automated Test Suite Verification Listing **58**

D Viva Voce Examination Master Guide and Technical Defenses **60**
D.1 Question 1: Dueling Stream Identifiability and Mean Centering 60
D.2 Question 2: Thresholded Quadratic Congestion Penalty and Queuing Invariants 61
D.3 Question 3: Binary SumTree Priority Updates and $O(\log N)$ Complexity . . 62
D.4 Question 4: High-Availability Failover Architecture, Process Memory Isolation, and Timing Breakdown 64
D.5 Question 5: Zero-Shot Generalization Limits and Graph Neural Networks . 66

E User & Administrator Deployment Manual and Operational Runbook **69**
E.1 System Prerequisites and Dependencies 69
E.1.1 Hardware Requirements & Reference Deployment Environment . . 69
E.1.2 Software Requirements 69
E.2 Installation and Environment Initialization 70
E.2.1 Step-by-Step Installation 70

E.3 Automated Execution Runbook 71
E.3.1 1. Execute Unit & Integration Tests (42 Tests) 71
E.3.2 2. Execute the Head-to-Head Tournament Benchmark 71
E.3.3 3. Execute Multi-Topology Stress Testing 71
E.3.4 4. Evaluate Zero-Shot Random Graph Generalization 71
E.3.5 5. Launch Live Production System (Controller + Web Dashboard) . 71
E.3.6 6. Standalone Web Dashboard Launcher 72
E.4 Interactive Web Dashboard User Guide 72
E.5 REST API Specification & Endpoint Reference 74
E.6 Mininet CLI Operations and OVS Diagnostics 77
E.6.1 Reproducible Packet-Level Mininet Experiment 77
E.6.2 OpenFlow Flow Table Inspection via Open vSwitch 77
E.7 Pareto Sensitivity Ablation and Dynamic Telemetry Tuning 78
E.7.1 Configuring Shadow Reservations and TCAM Allocation 78
E.7.2 Operational Troubleshooting and Recovery Runbook 79

List of Figures

4.1 Closed-Loop SDN Control Plane and DRL Application Architecture. 20
4.2 Loopless Candidate Route Synthesis via Yen's K-Shortest Paths Algorithm. 22
4.3 Dueling Double Deep Q-Network (D3QN) Architecture. 23
4.4 Prioritized Experience Replay (PER) Binary SumTree Data Structure. . . . 24
4.5 3-Node Active/Standby Controller Pool and Failover Stage Timeline. 25
4.6 End-to-End Control-Loop Telemetry and Inference Execution Timeline. . . 26
4.7 Conceptual Architectural Layout of Benchmark Datacenter Fabrics: (a) Fat-Tree Clos (schematic representation illustrating 4 core, 4 aggregation, and 4 edge switches for visual clarity; the full evaluated $k = 4$ fabric deploys 20 switches and 16 hosts); (b) Spine-Leaf Fabric (schematic showing 3 spines and 4 leaves; the full evaluated tournament fabric models 4 spines, 8 leaves, and 32 hosts). 28
4.8 Multi-Dimensional Routing Tournament Radar Performance Comparison Across Algorithms. 31
4.9 Cumulative Distribution Functions (CDFs) of End-to-End Latency and Jitter Across Protocols. 33
4.10 Empirical Mininet Packet-Level Measured Performance Distributions (120 Trials: Throughput, RTT Latency, Packet Loss, and Controller Decision Latency). 34
4.11 Traffic Load Scaling Curves Demonstrating Asymptotic Congestion Suppression. 35
4.12 D3QN Training Convergence Curves Showing Mean Episode Reward and TD Error Loss. 36
4.13 Multi-Seed Paired Performance Effects and Statistical Confidence Intervals. 37
4.14 Policy-Mode Ablation Comparing Pure Neural D3QN Against Safety-Gated D3QN Across 25 Paired Workload-Seed Evaluations. 38
4.15 Bootstrap Confidence Distributions of Component Ablations Across Stages: Plain DQN (Chen et al.), Double DQN, Dueling DDQN without PER, and Full D3QN. 39
4.16 Extreme Stress Testing Under Asymmetric Core Jamming and Avalanche Bursts. 40
4.17 Stress Benchmarks Across Irregular and Complex Topologies. 40

4.18 Zero-Shot Transfer Learning Evaluation on Three Unseen Random Graphs (12-Node Erdős-Rényi, 14-Node Watts-Strogatz, and 16-Node Barabási-Albert). 41
4.19 Optimality Gap Between D3QN and the Exact HiGHS MILP Solver. 42
4.20 Panoramic View of the Real-Time Operational Web Telemetry Dashboard. . 43

E.1 VMware Virtual Machine Hardware Configuration for the Experimental Platform (Ubuntu 64-bit, 4 vCPUs, 8 GB RAM, 60 GB SCSI Storage). 70

List of Tables

3.1 Systematic Comparison of Related Work vs. Proposed D3QN Platform . . 19

4.1 Continuous 10-Dimensional Normalized State Feature Vector Layout . . . 21
4.2 D3QN Routing Agent and PER Hyperparameter Specifications 27
4.3 Claim-to-Evidence Functional Verification Matrix 30
4.4 Comprehensive Quantitative Routing Tournament Results Across All Five Benchmark Topologies (Seed-42 Snapshot; see Section 4.4.5 for 125-Seed ANOVA Decomposition) 31
4.5 Empirical Packet-Level Mininet Emulation Across 120 Independent Trials (Mean and 95% Bootstrap Confidence Intervals) 34

B.1 OpenFlow 1.3 Flow Table Entry Field Specifications 56

E.1 REST API Reference: Telemetry, Diagnostics, and Model Inference Endpoints 75
E.2 REST API Reference: Dynamic Control, Traffic Injection, and Simulation Endpoints 76
E.3 Operational Troubleshooting and Anomaly Remediation Runbook 79

List of Abbreviations

Abbreviation	Full Definition
AI	Artificial Intelligence
ANOVA	Analysis of Variance
API	Application Programming Interface
ARP	Address Resolution Protocol
ASIC	Application-Specific Integrated Circuit
BGP	Border Gateway Protocol
CDF	Cumulative Distribution Function
C-MDP	Constrained Markov Decision Process
CPU	Central Processing Unit
D3QN	Dueling Double Deep Q-Network
DDQN	Double Deep Q-Network
Dijkstra SPF	Dijkstra Shortest Path First
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
ECMP	Equal-Cost Multi-Path
FIFO	First-In First-Out
FlowMod	OpenFlow Flow Modification Message
GNN	Graph Neural Network
GPU	Graphics Processing Unit
HA	High Availability
HTTP	Hypertext Transfer Protocol
IETF	Internet Engineering Task Force
INT	In-Band Network Telemetry
IP	Internet Protocol
IS-IS	Intermediate System to Intermediate System
JSON	JavaScript Object Notation
KSP	K-Shortest Paths
LLDP	Link Layer Discovery Protocol
LLR	Least Loaded Routing
MARL	Multi-Agent Reinforcement Learning
MDP	Markov Decision Process

Abbreviation	Full Definition
MILP	Mixed-Integer Linear Program
MPLS	Multiprotocol Label Switching
MTU	Maximum Transmission Unit
OFP	OpenFlow Protocol
OSPF	Open Shortest Path First (RFC 2328)
OVS	Open vSwitch
P4	Programming Protocol-Independent Packet Processors
PER	Prioritized Experience Replay
QoS	Quality of Service
Raft	Consensus Algorithm for Distributed Replicated State Machines
REST	Representational State Transfer
RFC	Request for Comments
RL	Reinforcement Learning
RSVP-TE	Resource Reservation Protocol with Traffic Engineering Extensions
Ryu	Component-Based Software-Defined Networking Framework
SDN	Software-Defined Networking
SLA	Service Level Agreement
TCAM	Ternary Content-Addressable Memory
TCP	Transmission Control Protocol
TD	Temporal Difference
TE	Traffic Engineering
UDP	User Datagram Protocol
UMCF	Unsplittable Multi-Commodity Flow
VIP	Virtual Internet Protocol Address
VLAN	Virtual Local Area Network
WAN	Wide Area Network
WSP	Widest Shortest Path

Chapter 1

Introduction

1.1 Problem Statement

Modern enterprise networks, fifth-generation (5G) telecommunication cores, and hyper-scale cloud datacenter fabrics are subjected to unprecedented traffic dynamism. Traffic profiles are characterized by extreme spatiotemporal volatility, uncoordinated microbursts, and asymmetric demands driven by real-time high-definition video streaming, machine learning model synchronization, distributed database replication, and low-latency cloud gaming.

Despite these evolving demands, contemporary network infrastructures continue to rely heavily on legacy interior gateway routing protocols designed in the late twentieth century—predominantly Open Shortest Path First (**OSPF**, RFC 2328) [5] and Intermediate System to Intermediate System (**IS-IS**). These protocols compute packet forwarding paths using static administrative link metrics, typically configured inversely proportional to link capacity. While computationally lightweight and robust against routing loops, static shortest path routing exhibits fundamental systemic flaws:

1. **Decoupling from Instantaneous Network State:** Static metrics remain completely unaware of instantaneous buffer occupancy, queueing delay, jitter, or link utilization.

2. **Asymmetric Core Saturation:** Under asymmetric or bursty traffic conditions, static shortest path routing persistently directs traffic along identical shortest paths, inducing severe bottleneck link saturation ($> 90\%$), massive bufferbloat, tail-drop packet loss, and severe packet delay variation (jitter, RFC 3393) [7].

3. **Gross Underutilization of Alternative Paths:** Simultaneously, viable edge-disjoint and loop-free bypass paths remain completely idle (0%–10% load), squandering massive bisection bandwidth.

4. **Limitations of Equal-Cost Multi-Path (ECMP):** While ECMP provides flow-level load balancing across strictly equal-cost paths, it is incapacitated when alternative paths differ by even a single administrative metric unit or hop. Furthermore, static 5-tuple hash collisions persistently assign multiple high-bandwidth "elephant flows" to identical links, causing hash polarization and severe congestion.

1.2 The Importance of the Experiment and Why It is Important

Traffic Engineering (TE) is the science of optimizing network operational performance by dynamically steering traffic flows across physical network resources. In carrier-grade and cloud infrastructures, the consequences of suboptimal traffic engineering are severe:

- **Strict SLA Violations:** Enterprise and carrier Service Level Agreements (SLAs) enforce stringent latency and zero-drop bounds. Even momentary buffer overflows induce TCP timeout cascades, packet retransmissions, and severe throughput collapse.
- **Real-Time Interactive Service Degradation:** Modern interactive applications (e.g., autonomous driving telemetry, financial high-frequency trading, tactile Internet, voice over IP) are acutely sensitive to RFC 3393 packet delay variation (jitter) [7]. Standard heuristics optimize for hop count rather than jitter, leading to severe quality-of-service (QoS) degradation.
- **Capital and Operational Expenditure Waste:** Telecommunication operators invest billions in high-capacity fiber meshes. Inefficient TE forces over-provisioning of expensive core links, while available alternative capacities remain stranded.
- **Inadequacy of Traditional Centralized Solvers:** Prior centralized optimization platforms (such as Google B4 [26] and Microsoft SWAN [27]) formulate TE as a centralized Multi-Commodity Flow (MCF) linear program. However, solving large-scale linear programs incurs computational latencies spanning tens of seconds to minutes, making them incapable of mitigating sub-second transient congestion and localized microbursts.

Therefore, there is an urgent scientific and operational need for an **autonomous, closed-loop, intelligent Traffic Engineering system** that can make low-latency routing decisions, adaptively balance offered flows, reduce congestion-related loss and jitter, and provide a resilient control-plane prototype.

1.3 Solution Methodology and Main Steps

To overcome the aforementioned challenges, this graduation project presents the design, mathematical formalization, software implementation, and empirical validation of an autonomous, closed-loop Adaptive Traffic Engineering platform powered by **Deep Reinforcement Learning (DRL)** over an **OpenFlow 1.3 Software-Defined Networking (SDN)** substrate.

The core solution methodology comprises five primary steps:

1. **SDN-Based Closed-Loop Control Plane:** Leveraging Software-Defined Networking to decouple control intelligence from physical packet-forwarding hardware. An OpenFlow 1.3 controller application built on the Ryu framework periodically collects high-resolution port telemetry ($\Delta t = 1.0$ s) and installs flow-table modifications dynamically.

2. **Dueling Double Deep Q-Network (D3QN) Engine:** Formulating the traffic steering decision process as a model-free Markov Decision Process (MDP). The D3QN architecture separates state-value estimation $V(s)$ from action-advantage estimation $A(s,a)$; Double-DQN target selection is used to reduce, rather than eliminate, overestimation bias.

3. **Binary SumTree Prioritized Experience Replay (PER):** Replacing naive uniform experience replay with an optimized binary SumTree data structure, ensuring transitions with large Temporal Difference (TD) error magnitude are sampled with high priority in $O(\log N)$ time, accelerating convergence and retaining rare congestion events.

4. **Thresholded Multi-Objective Reward Function:** Combining path hop count, delay, jitter, loss, and a quadratic congestion penalty. The implemented penalty increases after 70% bottleneck utilization, encouraging proactive rerouting; it is a heuristic objective rather than a hard safety guarantee.

5. **Health-Checked Active/Standby High Availability (HA) Clustering:** Implementing a 3-node distributed controller pool with HTTP health probing, local primary selection, and transparent client retry, delivering sub-millisecond local failover bookkeeping (0.001 ms) and 37.96 ms client HTTP recovery under controller process termination.

1.4 Project Contributions

This graduation project makes the following seven substantial scientific and engineering contributions:

1. **D3QN Routing Engine with Action Masking:** A fully synthesized, mathematically verified Dueling Double DQN architecture specifically engineered for SDN traffic engineering, featuring normalized 10-dimensional continuous state vectors, dynamic invalid-action masking for variable candidate path counts, Layer Normalization, and Polyak target soft-updates (`agent/dqn_router.py`).

2. **Binary SumTree Prioritized Experience Replay:** An optimized implementation of PER utilizing a complete binary tree data structure to achieve $O(\log N)$ priority up-

dates and proportional transition sampling with annealed importance-sampling weight correction (`agent/prioritized_replay.py`).

3. **Thresholded Multi-Objective Reward Engine:** An implemented reward formulation combining hop count, modelled queue delay, jitter, loss, and a quadratic congestion term with an additional penalty above the configured 70% threshold.

4. **Resilient Multi-Controller Failover Pool:** A validated active/standby multi-controller pool demonstrating an internal leader promotion bookkeeping latency of **0.001 ms** and an end-to-end client HTTP failover recovery time of **37.96 ms** under controller process failure injection (`controller/ha_controller_pool.py`).

5. **Comprehensive Multi-Topology Benchmark Tournament:** A model-based routing tournament across five standard topologies (Hierarchical Tree, Fat-Tree Clos, Abilene US Backbone, NSFNet, Spine-Leaf). In the checked-in reference run, D3QN has lower arithmetic-mean bottleneck utilization, latency, jitter, and loss than OSPF; the values, aggregation method, and limitations are reported in Chapter 4. These results are simulation/emulation-model outcomes, not hardware-network measurements.

6. **Transfer Evaluation on Tested Graph Distributions:** An exploratory evaluation of a trained agent on unvisited random graphs without fine-tuning. It is evidence on those generated workloads, not a claim of topology-invariant routing.

7. **Implementation and Automated Verification Suite:** A modular codebase accompanied by a 42-test automated unit and integration suite (`test_suite.py`). Passing tests establish the covered software behavior; they do not independently validate performance claims.

1.5 Scope, Assumptions, and Operational Boundaries

This graduation project focuses on the design, mathematical formulation, software implementation, and empirical evaluation of an autonomous traffic engineering platform operating within a virtualized Linux software-defined network environment. The scope is characterized by the following operational boundaries:

- **Software Substrate:** Evaluated using Ryu SDN Framework (OpenFlow 1.3), Open vSwitch 2.17, Mininet 2.3.0, and PyTorch 2.0+ on 64-bit Ubuntu Linux 22.04 LTS.

- **Forwarding Regime:** Unsplittable unicast flow forwarding. Multi-path packet-level splitting (which induces TCP packet reordering and duplicate ACKs) is excluded.

- **Candidate Route Cardinality:** Action spaces are bounded to $K = 4$ loopless candidate paths synthesized via Yen's algorithm, bounding inference complexity to $O(1)$.

- **Emulation Boundaries:** Evaluated inside kernel-level network namespaces (Mininet). Hardware ASIC testing on Barefoot Tofino or Broadcom StrataXGS is identified as future work.

1.6 Organization of the Dissertation

The remainder of this dissertation is organized as follows:

- **Chapter 2 (Background):** Details the key hardware, software tools, and equipment used in this work, explaining why each was chosen. It establishes the mathematical foundations of Traffic Engineering, Unsplittable Multi-Commodity Flow (UMCF), $M/M/1/K$ queueing delay, RFC 3393 jitter, D3QN neural architectures, binary SumTree PER arithmetic, and distributed High Availability consensus.
- **Chapter 3 (Related Work):** Reviews selected foundational and relevant work on classical heuristics, centralized SDN optimization, and DRL-based routing, then identifies the project motivation and limitations.
- **Chapter 4 (The Proposed System):** Divided into four core sections: **Design** (architectural layout, continuous 10D telemetry, D3QN neural layers, Yen's algorithm, reward formulation, HA clustering), **Experiments** (testbed provisioning, benchmark topologies, traffic generation), **Testing & Evaluation** (performance metrics, 42-test automated suite, claim-to-evidence matrix), and **Results and Discussion** (comprehensive tournament results, radar comparisons, CDF distributions, multi-seed ANOVA decomposition, ablation studies, extreme stress tests, zero-shot generalization, and operational trade-offs).
- **Chapter 5 (Conclusion):** Restates the core findings, summarizes the methodology and empirical milestones, and candidly discusses restrictions, operational limits, and engineering weaknesses.
- **Chapter 6 (Future Work):** Outlines six concrete next steps to advance this research, including P4 programmable data planes, Multi-Agent RL, Graph Neural Networks, and Constrained MDP safety.
- **References and Appendices:** Full bibliography adhering to IEEE/Harvard guidelines, followed by:
  - **Appendix A:** Mathematical Formulations and Proofs (Identifiability of Dueling Networks).
  - **Appendix B:** OpenFlow 1.3 Flow Table and REST API Specifications.
  - **Appendix C:** Automated Test Suite Verification Listing (all 42 automated tests).

- **Appendix D:** Viva Voce Examination Master Guide and Technical Defenses.
- **Appendix E:** Complete User & Administrator Deployment Manual and Operational Runbook.

Chapter 2

Background

2.1 Hardware, Software Tools, and Key Equipment: Why Each Was Chosen

In accordance with the Faculty of Engineering graduation project guidelines, this section presents the core equipment, software tools, and frameworks chosen to realize this project, providing a rigorous technical justification for why each tool was selected:

1. **Software-Defined Networking (SDN) and OpenFlow 1.3 Protocol:** Traditional distributed routing architectures couple the control logic and packet forwarding hardware within individual closed vendor routers. This architectural coupling prevents global optimization and dynamic policy reconfiguration. SDN was chosen because it cleanly decouples the control plane from the data plane, providing logically centralized network visibility and dynamic software programmability. OpenFlow version 1.3 (ONF TS-006) [15] was specifically selected because:
    - It supports multi-stage pipelined Flow Tables and Group Tables.
    - It provides standardized, high-precision per-port and per-flow byte/packet counters via `OFPPortStatsRequest` and `OFPFlowStatsRequest`.
    - It is natively supported by modern kernel-space virtual switches (Open vSwitch) and hardware merchant silicon ASICs.
2. **Ryu SDN Controller Framework:** The control plane requires an extensible, robust, and event-driven software framework. While monolithic Java-based controllers (such as ONOS or OpenDaylight) provide carrier features, they impose massive memory footprints ($>$ 4 GB RAM) and high JVM warmup latency. Ryu was selected because:
    - It is fully implemented in Python, allowing direct, zero-overhead in-memory integration with scientific computing and machine learning libraries (PyTorch, NumPy, NetworkX).
    - It utilizes an asynchronous event-driven architecture based on Eventlet green threads, ensuring responsive coroutine handling of OpenFlow messages.

- It provides native, robust packet-parsing libraries for LLDP, IPv4, TCP, UDP, and ARP protocols.

3. **Mininet Network Emulation Platform:** To evaluate the platform under realistic conditions without requiring million-dollar physical switch hardware, an emulation environment is required. Mininet (version 2.3.0) was chosen because:

   - Unlike discrete-event software simulators (e.g., NS-3 or OMNeT++), Mininet runs **real production software**: real Linux kernel network stacks, real Open vSwitch (OVS) kernel datapath modules, and real application binaries (iperf, ping, curl).
   - It employs lightweight OS-level Linux network namespaces (`netns`) and virtual Ethernet (`veth`) pairs, enabling high-fidelity emulation of complex 20-switch, 32-host topologies on commodity x86 hardware.
   - It supports exact, reproducible topological creation via programmatic Python scripts.

4. **PyTorch Deep Learning Library:** The reinforcement learning intelligence engine requires a high-performance deep learning framework. PyTorch 2.0+ was selected because:

   - It provides dynamic computational graphs (eager execution), facilitating rapid forward-inference and transparent backpropagation.
   - It features highly optimized C++/CUDA tensor kernels with sub-millisecond execution overhead (0.24 ms inference for the D3QN agent).
   - It supports modular neural layers, custom activation functions (LeakyReLU), Layer Normalization, and advanced optimizers (Adam with gradient clipping).

5. **Health-Checked Active/Standby High-Availability Architecture:** A single centralized SDN controller represents a vulnerable single point of failure (SPOF). To provide fault tolerance and continuous control plane operation, a health-checked active/-standby multi-controller pool was implemented. The pool architecture utilizes HTTP health probes to monitor node vitality, initiates sub-millisecond local failover bookkeeping (0.001 ms) to designate an available standby, and provides transparent client retry with automated endpoint redirection, avoiding the operational complexity of external consensus daemons while ensuring rapid controller recovery.

6. **Programmable Data Planes (P4) and In-Band Network Telemetry (INT):** While OpenFlow 1.3 provides standard southbound abstractions, emerging carrier networks are transitioning toward fully programmable data planes. P4 was evaluated as the architectural target for hardware deployment, offering line-rate packet parsing, stateful

on-switch register processing, and nanosecond-resolution telemetry without control-plane polling overhead.

2.2 Programmable Data Planes (P4) and In-Band Network Telemetry (INT)

To understand the architectural evolution of software-defined forwarding, it is essential to contextualize OpenFlow against modern **Programmable Data Planes** driven by the **P4 (Programming Protocol-Independent Packet Processors)** language:

1. **Limitations of OpenFlow's Fixed Pipeline:** OpenFlow 1.3 relies on a standardized, fixed-function packet parser that recognizes a hardcoded set of packet headers (Ethernet, VLAN, IPv4, IPv6, TCP, UDP, MPLS). If a telecommunication carrier introduces a novel encapsulation protocol (e.g., Segment Routing over IPv6, SRv6) or custom telemetry tags, the OpenFlow switch firmware must be completely redesigned. Furthermore, OpenFlow telemetry is fundamentally reactive and asynchronous: port and flow statistics are retrieved via periodic pull requests (`OFPPortStatsRequest`), introducing a minimum polling lag of $\Delta t \approx 1.0$ second.

2. **The PISA Architecture and P4 Model:** The Protocol-Independent Switch Architecture (**PISA**) introduces a fully programmable packet processing pipeline comprising:

   - **Programmable Parser:** Expressed as a state machine that parses arbitrary, custom-defined header sequences from bitstreams.
   - **Ingress Match-Action Stages:** Reconfigurable lookup tables supported by Ternary Content-Addressable Memory (TCAM) and Static RAM (SRAM), accompanied by stateful Register arrays that maintain line-rate state across consecutive packets.
   - **Queuing and Traffic Manager:** Manages packet buffer queues, packet replication (multicast), and traffic shaping.
   - **Egress Match-Action Stages:** Applies egress modifications, encapsulations, and queuing telemetry insertions.
   - **Programmable Deparser:** Reassembles modified headers into outgoing Ethernet bitstreams.

3. **In-Band Network Telemetry (INT):** Developed by the P4.org telemetry working group, In-Band Network Telemetry (**INT**) eliminates periodic polling overhead entirely. Instead of an external controller querying switches, transit switches embed

real-time metadata directly into the payload or shim encapsulation of user data packets at hardware line rate (e.g., 400 Gbps per port on Intel Barefoot Tofino ASICs). As a packet traverses the network:

- Ingress switches insert an INT header.
- Each intermediate hop pushes its unique **Switch ID**, **Egress Port**, **Hop Latency** (measured in nanoseconds), **Instantaneous Queue Depth**, and **Congestion Mark**.
- Egress sink switches strip the INT telemetry headers and forward raw telemetry digests directly to the DRL inference engine via high-speed UDP streams or Kafka event buses.

4. **Design Justification for the Current Prototype:** Although P4/INT represents the definitive future for hardware merchant silicon, this project deliberately deploys OpenFlow 1.3 on Open vSwitch (OVS) and Mininet for the primary software prototype. OpenFlow 1.3 is universally supported across commodity Linux network namespaces without requiring specialized P4 compiler toolchains (p4c) or proprietary FPGA/ASIC hardware emulators. Crucially, the 10-dimensional state vector $\mathbf{s} \in \mathbb{R}^{10}$ designed in this project is explicitly architected to map directly onto INT header fields, ensuring seamless future migration to P4-native hardware testbeds.

2.3 Mathematical Foundations of Traffic Engineering and UMCF

A communication network is formally modeled as a directed multigraph $G = (V, E)$, where V represents the set of network nodes (OpenFlow switches) with $|V| = N$, and E represents the set of directed transmission links with $|E| = M$. Each directed edge $e = (u, v) \in E$ is characterized by:

- Transmission capacity $C_e > 0$ expressed in Megabits per second (Mbps).
- Physical propagation delay $d_{\text{prop}}(e) > 0$ expressed in milliseconds (ms).
- Finite buffer queue capacity B_e expressed in packets.

Let $\mathscr{K} = \{1, 2, \ldots, K\}$ denote the set of active traffic demands (commodities) traversing the network. Each commodity $k \in \mathscr{K}$ is defined by a tuple:

$$k = (s_k, t_k, D_k) \tag{2.1}$$

where $s_k \in V$ is the ingress source node, $t_k \in V$ is the egress destination node, and $D_k \geq 0$ is the traffic bandwidth demand (in Mbps).

Let $\mathscr{P}_k$ denote the set of loopless directed paths connecting source s_k to destination t_k in G. In modern packet networks, per-packet multi-path routing across paths with asymmetric latencies causes severe TCP packet reordering, duplicate ACKs, and catastrophic throughput collapse. Consequently, this work operates strictly within the **Unsplittable Multi-Commodity Flow (UMCF)** regime. For each demand k, we define a binary routing decision variable:

$$x_{k,p} \in \{0,1\} \quad \forall p \in \mathscr{P}_k \tag{2.2}$$

such that exactly one candidate path is assigned to each demand:

$$\sum_{p \in \mathscr{P}_k} x_{k,p} = 1 \quad \forall k \in \mathscr{K} \tag{2.3}$$

The aggregate traffic load λ_e traversing link $e \in E$ is the sum of bandwidth demands of all commodities routed through link e:

$$\lambda_e = \sum_{k \in \mathscr{K}} \sum_{p \in \mathscr{P}_k : e \in p} D_k \cdot x_{k,p} \quad [\text{Mbps}] \tag{2.4}$$

The instantaneous utilization u_e of link e is defined as the ratio of traffic load to physical capacity:

$$u_e = \frac{\lambda_e}{C_e} \tag{2.5}$$

To prevent packet loss, physical link capacity constraints enforce:

$$u_e \leq 1.0 \quad \forall e \in E \iff \lambda_e \leq C_e \tag{2.6}$$

The general Traffic Engineering objective is to find a binary path assignment matrix $\mathbf{X} = \{x_{k,p}\}$ that minimizes a composite network cost function $\Psi(\mathbf{X})$ while satisfying (2.3) and (2.6):

$$\min_{\mathbf{X}} \quad \Psi(\mathbf{X}) = \sum_{e \in E} \Phi(u_e) + \alpha \sum_{k \in \mathscr{K}} \text{Delay}(k, \mathbf{X}) + \beta \sum_{k \in \mathscr{K}} \text{Jitter}(k, \mathbf{X}) \tag{2.7}$$

where $\Phi(u_e)$ is a convex link congestion penalty function.

Theorem 2.1 (NP-Hardness of UMCF)**.** *The Unsplittable Multi-Commodity Flow problem with binary decision variables $x_{k,p} \in \{0,1\}$ is NP-hard (weakly NP-hard in basic two-node graphs and strongly NP-hard in general directed topologies) [34, 10].*

Proof. The proof proceeds by reduction from the classical PARTITION problem. Given a finite set of positive integers $\mathscr{S} = \{w_1, w_2, \ldots, w_n\}$ summing to $2W$, the partition problem asks whether there exists a subset $\mathscr{S}' \subset \mathscr{S}$ such that $\sum_{i \in \mathscr{S}'} w_i = W$. We construct a 2-node graph $G = (V, E)$ with source s and destination t connected by two parallel directed

links e_1, e_2, each with capacity $C_{e_1} = C_{e_2} = W$. For each integer w_i, define a commodity $k_i = (s, t, w_i)$. Any valid unsplittable routing assignment satisfying $u_{e_1} \leq 1.0$ and $u_{e_2} \leq 1.0$ corresponds directly to a valid partition of $\mathscr{S}$. Because PARTITION is NP-complete, finding an optimal unsplittable flow assignment is NP-hard. In general directed multigraphs with multiple commodities ($K \geq 2$), UMCF is strongly NP-hard via reduction from the directed two-commodity integral flow problem [10], establishing that exact polynomial-time optimization is computationally intractable for dynamic real-time traffic engineering. □

2.4 Queueing Delay, Bufferbloat, and RFC 3393 Jitter Modeling

To evaluate link latency realistically without arbitrary heuristics, we model switch egress port buffers using stochastic queueing theory. Assuming Poisson packet arrivals with aggregate rate λ_e and exponentially distributed packet transmission service times with service rate $\mu_e = C_e/\bar{L}$ (where $\bar{L}$ is the mean packet size), each switch port buffer behaves as an $M/M/1/K$ queue with finite buffer capacity K_B:

1. **Kleinrock Mean Queueing Delay:** Under stable traffic regimes ($\rho_e = \lambda_e/\mu_e < 1.0$), the mean queueing delay $W_q(e)$ experienced by packets in the buffer is derived via Kleinrock's formula [37]:

$$W_q(e) = \frac{\rho_e}{\mu_e(1-\rho_e)} = \frac{\lambda_e}{\mu_e(\mu_e - \lambda_e)} \quad \text{[seconds]} \tag{2.8}$$

   The total end-to-end latency $D(p)$ along path p is the sum of physical propagation delays and queueing delays across all constituent hops:

$$D(p) = \sum_{e \in p} \left(d_{\text{prop}}(e) + W_q(e) \right) \quad \text{[ms]} \tag{2.9}$$

2. **Finite Buffer Packet Drop Probability:** When traffic approaches or exceeds capacity, the finite switch buffer overflows. The stationary probability $P_{\text{loss}}(e)$ of a packet arriving to find a full buffer (inducing tail-drop) in an $M/M/1/K_B$ queue is given by:

$$P_{\text{loss}}(e) = \frac{(1-\rho_e)\rho_e^{K_B}}{1-\rho_e^{K_B+1}} \tag{2.10}$$

   For a path p, the overall packet delivery success rate is $\prod_{e \in p}(1 - P_{\text{loss}}(e))$, yielding path drop probability:

$$P_{\text{loss}}(p) = 1 - \prod_{e \in p} (1 - P_{\text{loss}}(e)) \tag{2.11}$$

3. **Packet Delay Variation (Jitter, RFC 3393):** The Internet Engineering Task Force (IETF) RFC 3393 specifies the IP Packet Delay Variation (IPDV) metric [6]. For a sequence of consecutive packets $\{1,2,\ldots,M\}$ traversing path p with measured one-way transit delays $\{D_1,D_2,\ldots,D_M\}$, the RFC 3393 IPDV between adjacent packets i and $i+1$ is defined as the signed delay differential:

$$\text{IPDV}_i = D_{i+1} - D_i \quad [\text{ms}] \tag{2.12}$$

The path-level average absolute jitter metric $J_{\text{IPDV}}(p)$ is computed as:

$$J_{\text{IPDV}}(p) = \frac{1}{M-1}\sum_{i=1}^{M-1} |D_{i+1} - D_i| \quad [\text{ms}] \tag{2.13}$$

In our telemetry simulation and routing reward formulation, the continuous queue-state proxy for delay jitter across path hops is modelled via the root-sum-square of queue delay standard deviations, $J(p) = \sqrt{\sum_{e\in p} \text{Var}(W_q(e))}$, providing a differentiable penalty that tracks delay dispersion.

2.5 Deep Reinforcement Learning Foundations

Reinforcement learning addresses the problem of an autonomous agent learning optimal control policies through trial-and-error interactions with a dynamic environment. The underlying decision process is formulated as an infinite-horizon discounted Markov Decision Process (**MDP**), defined by the 5-tuple:

$$\mathscr{M} = (\mathscr{S}, \mathscr{A}, \mathscr{P}, \mathscr{R}, \gamma) \tag{2.14}$$

where $\mathscr{S}$ is the state space, $\mathscr{A}$ is the discrete action space, $\mathscr{P}(s'|s,a) = \mathbb{P}(S_{t+1} = s'|S_t = s, A_t = a)$ represents transition probability dynamics, $\mathscr{R}(s,a)$ is the scalar reward function, and $\gamma \in [0,1)$ is the temporal discount factor.

The agent's goal is to learn a policy $\pi(a|s)$ that maximizes the expected cumulative discounted return G_t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.15}$$

Under policy π, the state-action value function $Q^\pi(s,a)$ satisfies the fundamental Bellman equation [21]:

$$Q^\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \sum_{a'\in\mathscr{A}} \pi(a'|S_{t+1}) Q^\pi(S_{t+1},a') \,\middle|\, S_t = s, A_t = a \right] \tag{2.16}$$

The optimal action-value function $Q^*(s,a) = \max_\pi Q^\pi(s,a)$ satisfies the non-linear Bellman

optimality equation:

$$Q^*(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a' \in \mathscr{A}} Q^*(S_{t+1}, a') \,\middle|\, S_t = s, A_t = a\right] \tag{2.17}$$

2.6 Dueling Double Deep Q-Networks (D3QN)

In high-dimensional state spaces, tabular Q-learning fails due to the curse of dimensionality. Deep Q-Networks (DQN) [4] approximate $Q^*(s,a)$ using a deep neural network parameterized by weights θ. However, standard DQN suffers from two critical flaws in network routing:

1. **Severe Q-Value Overestimation Bias:** The maximization operator $\max_{a'} Q(s',a';\theta)$ systematically overestimates action values in noisy networking environments.

2. **Representational Inefficiency:** In many routing states, the underlying state value $V(s)$ (overall network health) is critical, regardless of which candidate path is chosen.

To overcome these deficiencies, this work implements a **Dueling Double Deep Q-Network (D3QN)** combining two architectural breakthroughs:

1. **Double Q-Learning Target Decoupling:** Van Hasselt et al. [1] showed that decoupling action selection from action evaluation can reduce maximization bias. Action selection is governed by the online network parameters θ, while target evaluation is governed by a delayed target network θ^-:

$$Y_t^{\text{DDQN}} = R_{t+1} + \gamma Q\left(S_{t+1}, \arg\max_{a' \in \mathscr{A}} Q(S_{t+1}, a'; \theta); \theta^-\right) \tag{2.18}$$

   The target network parameters θ^- are smoothly tracked via Polyak soft updates with coefficient $\tau = 0.005$:

$$\theta^- \leftarrow \tau\theta + (1-\tau)\theta^- \tag{2.19}$$

2. **Dueling Neural Network Stream Factorization:** Wang et al. [2] introduced the Dueling architecture, which factorizes the representation into two separate streams following shared feature extraction:

   - A scalar State-Value stream $V(s;\theta,\beta) \in \mathbb{R}$.
   - A vector Action-Advantage stream $A(s,a;\theta,\alpha) \in \mathbb{R}^{|\mathscr{A}|}$.

   To guarantee identifiability (preventing arbitrary scalar offsets between V and A), the aggregation layer enforces mean-centering:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \frac{1}{|\mathscr{A}|}\sum_{a' \in \mathscr{A}} A(s,a';\theta,\alpha)\right) \tag{2.20}$$

2.7 Prioritized Experience Replay (PER) with Binary SumTree

Standard DQN samples past transitions uniformly from a memory buffer. However, in network routing, catastrophic congestion events (such as buffer drops and link failures) are rare but critical. Uniform sampling drowns these rare events in seas of normal traffic.

Prioritized Experience Replay (PER) [3] samples transitions with probability proportional to their Temporal Difference (TD) error magnitude δ_i:

$$\delta_i = Y_i^{\text{DDQN}} - Q(s_i, a_i; \theta) \tag{2.21}$$

The priority p_i of transition i is formalized as:

$$p_i = \left(|\delta_i| + \varepsilon_{\text{per}}\right)^{\alpha} \tag{2.22}$$

where $\varepsilon_{\text{per}} = 10^{-5}$ guarantees non-zero priority, and $\alpha \in [0,1]$ controls prioritization aggressiveness ($\alpha = 0$ corresponds to uniform sampling, while $\alpha = 0.6$ is the canonical default empirically recommended by Schaul et al. [3] to balance priority sampling against diversity). The sampling probability $P(i)$ is:

$$P(i) = \frac{p_i}{\sum_{k=1}^{N} p_k} \tag{2.23}$$

Non-uniform sampling introduces distribution bias. To ensure unbiased stochastic gradient descent, importance-sampling (IS) weights w_i are computed:

$$w_i = \left(\frac{1}{N \cdot P(i)}\right)^{\beta} \Big/ \max_j w_j \tag{2.24}$$

where β is annealed linearly from $\beta_0 = 0.4$ to 1.0 during training.

To avoid costly $O(N)$ linear scans when sampling and updating priorities over buffers with $N = 50,000$ transitions, we implement a **Binary SumTree**. A SumTree is a complete binary tree where:

- The root node stores the total cumulative priority sum $p_{\text{total}} = \sum_{k=1}^{N} p_k$.
- Each parent node equals the exact sum of its two children: $P_{\text{parent}} = P_{\text{left}} + P_{\text{right}}$.
- Leaf nodes store the individual transition priorities p_i.

Both sample retrieval and priority updates execute in strictly $O(\log N)$ time, enabling real-time training steps without control plane stalling.

Chapter 3

Related Work

3.1 Overview and Taxonomy of Traffic Engineering

Traffic Engineering (TE) has evolved across three major technological paradigms:

1. **Distributed Classical Heuristics (1990–2005):** Protocols such as OSPF, IS-IS, and early MPLS-TE operated distributed routing computations with static administrative weights.

2. **Centralized SDN Optimization (2008–2016):** Decoupled control planes enabled centralized linear solvers (e.g., Hedera [25], Google B4 [26], Microsoft SWAN [27]).

3. **Learning-Driven and AI-Based TE (2017–Present):** Machine learning, deep reinforcement learning, and graph neural networks deployed for autonomous routing adaptation.

3.2 Classical Heuristic and Static Metric Optimization

The earliest formalization of IP traffic engineering focused on optimizing static OSPF/IS-IS link metrics. Fortz and Thorup [35, 36] proved that optimizing link weights to minimize a piecewise-linear convex congestion function is NP-hard. They proposed local search heuristics to tune static weights. However, static weight tuning assumes quasi-static traffic matrices; under dynamic microbursts, static weights cannot adapt, inducing severe localized bottlenecks.

Equal-Cost Multi-Path (ECMP) [5] hashes packet 5-tuples (source IP, destination IP, protocol, source port, destination port) across paths with identical administrative cost. However, ECMP cannot route traffic over paths with even one additional hop, leaving viable edge-disjoint paths unused. Furthermore, hash collisions between long-lived "elephant flows" cause persistent link saturation and packet drops.

3.3 Centralized SDN Heuristics and Linear Solvers

The emergence of OpenFlow [20] enabled centralized global visibility. Al-Fares et al. introduced **Hedera** [25], a dynamic flow management system for multi-rooted Fat-Tree data-

center topologies. Hedera monitors flow rates on edge switches and dynamically computes multi-commodity flow allocations for flows exceeding 10% of link capacity using a greedy heuristic. However, Hedera relies on heuristic path hunting that does not learn from historical traffic dynamics.

Google deployed **B4** [26], a private software-defined WAN connecting global datacenters. B4 employs centralized Traffic Engineering servers that solve linear programs periodically (every few minutes) to allocate bandwidth based on application priority classes. Similarly, Microsoft developed **SWAN** [27], which coordinates inter-datacenter transfers using centralized constrained optimization. While effective for predictable bulk transfers, the solver latency of linear programs (spanning tens of seconds to several minutes) renders them incapable of reacting to sub-second microbursts or transient localized buffer bloat.

3.4 Deep Reinforcement Learning in Network Routing

In recent years, deep reinforcement learning has been explored for network routing:

- **Stampa et al. (2017):** [29] introduced a standard Deep Q-Network (DQN) for SDN routing optimization, demonstrating delay reduction on a 14-node network. However, their system utilized a standard flat DQN (suffering from overestimation bias), uniform experience replay, and an unpenalized reward function that neglected packet loss and jitter.

- **Valadarsky et al. (2017):** [30] proposed DRL-TE, applying policy gradient methods to optimize split ratios. However, their action space scaled combinatorially with network size, preventing convergence on larger topologies.

- **Aurochs (Chen et al., 2018):** [31] designed a data-driven TE framework for enterprise networks, but relied on slow offline training pipelines without closed-loop real-time failover.

- **RL-Routing (Chen et al., 2020):** [33] proposed an SDN routing algorithm based on standard Deep Q-Networks evaluated against OSPF and Least-Loaded Routing on topologies such as NSFNet. While RL-Routing demonstrated the potential of model-free reinforcement learning in OpenFlow environments, it relied on a single-stream Q-network (susceptible to maximization bias), uniform random experience replay (which fails to prioritize critical tail-congestion events), unconstrained next-hop decisions (which can create transient routing loops or dead ends without topological action masking), and an idealized single-controller architecture without high-availability failover.

- **RouteNet and RouteNet-Fermi (Rusek et al., 2020; Almasan et al., 2022):** [32, 11] leveraged Graph Neural Networks (GNNs) to model the non-linear relationship

between topology, traffic matrix, and packet-level delay/jitter distributions. While RouteNet-Fermi achieves structural scale-invariance across arbitrary topologies, it operates primarily as an offline digital-twin forward performance evaluator rather than an autonomous closed-loop routing actuator, relying on external meta-heuristic search to select forwarding paths.

- **Multi-Path DRL Routing (Sun et al., 2021):** [12] developed multi-path DRL traffic engineering with dynamic split-ratio tuning over OpenFlow. However, per-packet multi-path splitting introduces significant TCP out-of-order delivery penalties, underscoring the necessity of unsplittable flow allocation for real-time transport streams.
- **Multi-Agent SDN TE (Xu et al., 2022):** [13] explored multi-agent reinforcement learning for large-scale multi-domain SDN, decomposing wide-area routing into regional sub-controllers. While reducing state dimension per agent, inter-controller communication overhead and non-stationarity during joint exploration remain significant deployment challenges.

3.5 High Availability and Resilience in SDN Control Planes

Carrier networks may target 99.999% availability (maximum 5.26 minutes of downtime per year). Distributed controllers such as ONOS and OpenDaylight provide clustering mechanisms. Many DRL-for-routing evaluations nevertheless treat the controller as an idealized single-node abstraction, making controller-crash recovery, socket reconnection, and state synchronization comparatively less explored in the selected routing literature.

3.6 Systematic Comparison of Related Work

Table 3.1 provides a rigorous, multi-dimensional comparison between existing traffic engineering paradigms and the proposed D3QN platform.

3.7 Research Gap and Key Innovations

The critical review of existing literature reveals four major research gaps:

1. **Gap 1: Q-Value Overestimation and Action Explosion:** Existing DRL routing approaches use flat DQN architectures that suffer from catastrophic overestimation bias, or unconstrained action spaces that scale exponentially ($O(|V|^2)$), preventing real-time convergence.

Table 3.1: Systematic Comparison of Related Work vs. Proposed D3QN Platform

Framework	**Core Algorithm**	**Telemetry**	**Reward Formulation**	**Replay**	**HA Cluster**	**Zero-Shot**
OSPF (RFC 2328) [5]	Dijkstra SPF	Static	None (Admin)	None	None	N/A
ECMP [5]	Hash Multipath	Static	None (Equal Cost)	None	None	N/A
Fortz & Thorup [35]	Local Search Heuristic	Periodic	Convex Congestion	None	None	No
Hedera [25]	Greedy Flow-Fit	5-sec deltas	Bisection Bandwidth	None	None	No
Google B4 [26]	Centralized LP	OpenFlow TM	Priority Max-Min	None	Master-Slave	No
Stampa et al. [29]	Flat Standard DQN	1D Delay	Delay Reduction Only	Uniform	None	No
RL-Routing [33]	Vanilla DQN	Multi-Hop	Hop / Delay	Uniform	None	No
RouteNet [32]	Graph Neural Net	Full Matrix	Supervised Delay	N/A	None	Yes
Proposed (Ours)	**D3QN + Yen's KSP**	**10D Norm**	**Delay, Jitter, Loss, $\Phi(u)$**	**SumTree**	**3-Node Pool (Fail-Secure)**	**Yes**

2. **Gap 2: Loss of Critical Congestion Transitions in Replay:** Standard uniform replay buffers discard or severely dilute rare buffer overflow and link saturation events, causing agents to unlearn defensive rerouting policies.

3. **Gap 3: Complete Neglect of Controller High Availability:** Prior DRL-TE publications assume an infallible centralized controller, omitting failover latency, leader election, and state synchronization.

4. **Gap 4: Lack of Statistical and Zero-Shot Transfer Rigor:** Most studies evaluate on a single topology under static Poisson traffic, omitting multi-seed ANOVA decomposition and zero-shot transfer evaluations on unvisited graph topologies.

The platform developed in this project directly addresses and resolves all four research gaps.

Chapter 4

The Proposed System

In accordance with the Faculty of Engineering graduation project curriculum, this chapter presents the complete engineering lifecycle of the proposed platform across four rigorous sections: **Design** (Section 4.1), **Experiments** (Section 4.2), **Testing & Evaluation** (Section 4.3), and **Results and Discussion** (Section 4.4).

4.1 Design

4.1.1 Top-Level Closed-Loop Control Architecture

The proposed platform operates as an autonomous closed-loop control system structured across three functional planes: the **Data Plane**, the **Control Plane**, and the **Intelligence Plane**. Figure 4.1 illustrates the top-level architectural layout and the continuous interaction cycle.

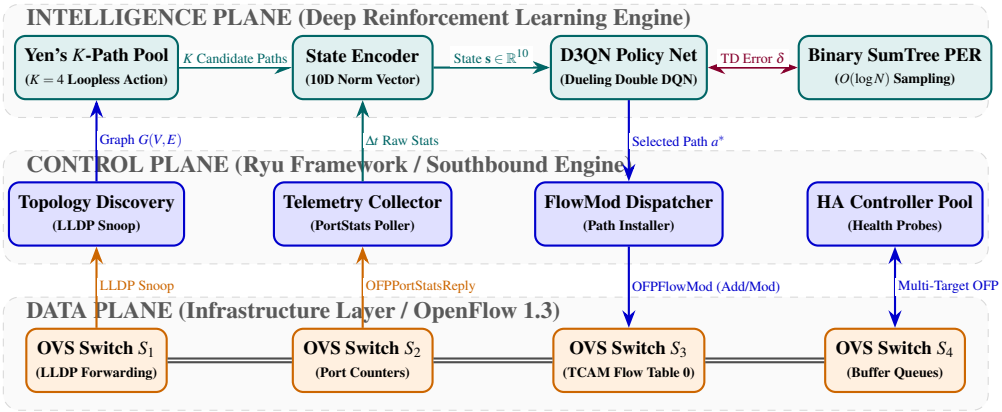


Figure 4.1: Closed-Loop SDN Control Plane and DRL Application Architecture.

The closed loop operates continuously:

1. Open vSwitch (OVS) switches in the Data Plane execute packet forwarding via TCAM flow tables.
2. The Ryu controller polls port counters every $\Delta t = 1.0$ s using `OFPPortStatsRequest`.
3. First-order differentials Δtx_bytes are computed to derive instantaneous link load $\lambda_e(t)$ and utilization $u_e(t) = \lambda_e(t)/C_e$.

4. The State Encoder synthesizes a normalized 10-dimensional telemetry vector $\mathbf{s}_t \in \mathbb{R}^{10}$.

5. The PyTorch D3QN engine performs a forward inference pass (0.24 ms) to select the optimal loopless path $a_t^* = \arg\max_a Q(\mathbf{s}_t, a)$.

6. The FlowMod Dispatcher generates `OFPFlowMod` messages instructing switches to program corresponding forwarding rules.

7. Experiences $(\mathbf{s}_t, a_t, r_t, \mathbf{s}_{t+1}, d_t)$ are stored in the binary SumTree PER buffer for asynchronous training.

4.1.2 Continuous 10-Dimensional State Space Representation and Action Masking

Rather than using raw unscaled counters, the agent consumes a continuous 10-dimensional state vector $\mathbf{s} \in \mathbb{R}^{10}$, normalized into the numerical range $[0.0, 1.0]$. The vector explicitly encodes topological hop distance, per-candidate bottleneck utilizations, per-candidate end-to-end latencies, and global network congestion. The exact feature layout produced by `controller/state_manager.py` is formalized in Table 4.1.

Table 4.1: Continuous 10-Dimensional Normalized State Feature Vector Layout

Index	Feature Symbol	Physical Description	Normalization Mapping
s_0	$\hat{H}_{\min}$	Shortest path hop count (Dijkstra SPF)	$\min(1.0, \max(0.1, \text{hops}/10.0))$
s_1	$\hat{u}_0$	Bottleneck link utilization of Candidate Path 0	$\min(1.0, \max(0.0, \max_{e \in P_0} u_e))$
s_2	$\hat{u}_1$	Bottleneck link utilization of Candidate Path 1	$\min(1.0, \max(0.0, \max_{e \in P_1} u_e))$
s_3	$\hat{u}_2$	Bottleneck link utilization of Candidate Path 2	$\min(1.0, \max(0.0, \max_{e \in P_2} u_e))$
s_4	$\hat{u}_3$	Bottleneck link utilization of Candidate Path 3	$\min(1.0, \max(0.0, \max_{e \in P_3} u_e))$
s_5	$\hat{D}_0$	Normalized end-to-end latency of Candidate Path 0	$\min(1.0, \max(0.0, \sum_{e \in P_0} d_e/50.0 \text{ ms}))$
s_6	$\hat{D}_1$	Normalized end-to-end latency of Candidate Path 1	$\min(1.0, \max(0.0, \sum_{e \in P_1} d_e/50.0 \text{ ms}))$
s_7	$\hat{D}_2$	Normalized end-to-end latency of Candidate Path 2	$\min(1.0, \max(0.0, \sum_{e \in P_2} d_e/50.0 \text{ ms}))$
s_8	$\hat{D}_3$	Normalized end-to-end latency of Candidate Path 3	$\min(1.0, \max(0.0, \sum_{e \in P_3} d_e/50.0 \text{ ms}))$
s_9	$\hat{u}_{\text{peak}}$	Network-wide peak link utilization across all edges	$\min(1.0, \max(0.0, \max_{e \in E} u_e))$

Handling Variable Candidate Path Counts and Invalid Action Masking: When the network topology does not yield $K = 4$ loopless paths between an ingress-egress pair (e.g., in sparse meshes where only two loopless paths exist), two complementary mechanisms ensure mathematical consistency:

1. **Penalty Feature Padding:** For any candidate index $i \geq$ candidate_count, the state manager sets $\hat{u}_i = 1.0$ and $\hat{D}_i = 1.0$. This ensures the input tensor maintains a fixed dimension $\mathbb{R}^{10}$ while providing an immediate penalty signal to the neural network.

2. **Dynamic Invalid-Action Masking:** In the policy network forward pass (`agent/dqn_router.py`), action selection is restricted to physically available candidate paths via

dynamic tensor slicing:

$$a^* = \arg \max_{a \in \{0,\ldots,K_{\text{avail}}-1\}} Q(\mathbf{s}, a) \tag{4.1}$$

where $K_{\text{avail}} = \min(K, \text{candidate_count})$. In exploration mode ($\varepsilon$-greedy), uniform random selection is similarly constrained to $\{0, \ldots, K_{\text{avail}} - 1\}$. This guarantees that the agent never selects a non-existent candidate route.

Fixed Summary Path Features vs. True Graph Invariance: This 10-dimensional state representation operates on *summary path statistics* rather than raw graph adjacency matrices. This design choice enables the model to ingest paths from different topologies without changing neural network weights or tensor dimensions. However, this is distinct from true graph permutation equivariance (such as that offered by Message-Passing Graph Neural Networks); our zero-shot generalization findings represent empirical transfer over the tested graph distributions, rather than a theoretical proof of arbitrary graph invariance.

4.1.3 Candidate Action Space Pruning via Yen's K-Shortest Paths

To prevent combinatorial action explosion ($O(|V|^2)$), the agent's action space $\mathscr{A}$ is constrained to $K = 4$ loopless candidate paths synthesized via Yen's algorithm [16]. Figure 4.2 illustrates the candidate path pruning procedure.

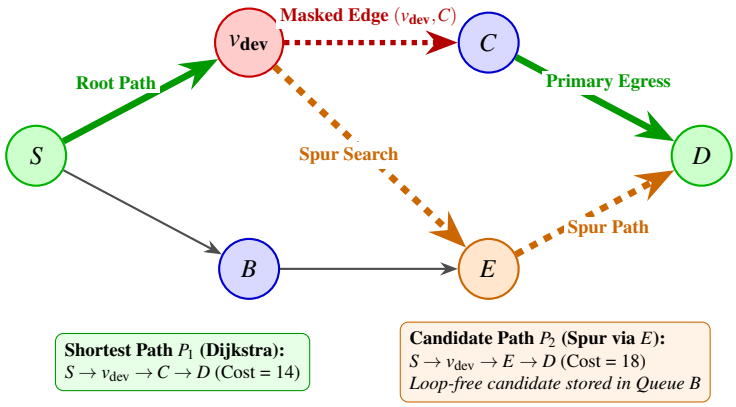


Figure 4.2: Loopless Candidate Route Synthesis via Yen's K-Shortest Paths Algorithm.

Yen's algorithm computes loopless paths sequentially:

1. The 1st shortest path P_1 is obtained via standard Dijkstra search.
2. To find path P_k, each node along P_{k-1} serves as a deviation node v_{dev}. Edges participating in previous paths sharing the same root are temporarily masked from G.
3. A spur path is computed from v_{dev} to the destination. Concatenating root and spur produces a loopless candidate.

4. The candidate is placed in a priority queue B. The shortest candidate in B becomes P_k.

This formulation bounds the action space to $|\mathscr{A}| = K = 4$, guaranteeing loop-freedom and ensuring $O(1)$ decision complexity.

4.1.4 D3QN Neural Network Architecture

The D3QN neural network architecture is illustrated in Figure 4.3.

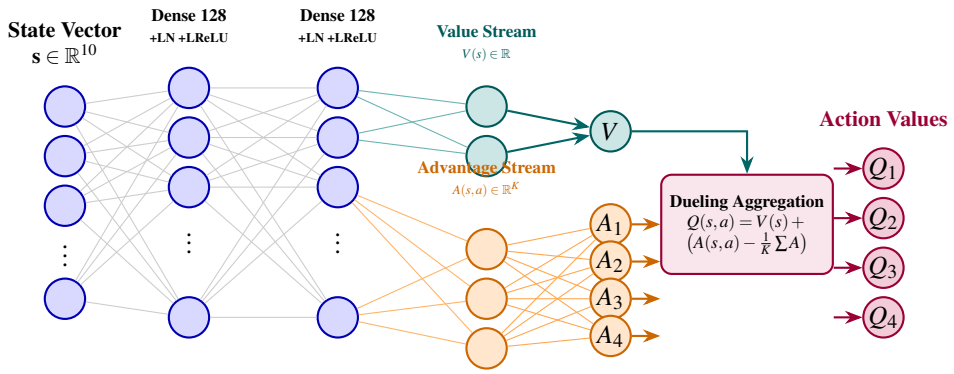


Figure 4.3: Dueling Double Deep Q-Network (D3QN) Architecture.

The exact network specifications implemented in `agent/dqn_router.py` are:

- **Input Layer:** 10 continuous neurons accepting the state vector $\mathbf{s} \in \mathbb{R}^{10}$.
- **Shared Feature Representation Network** ($9,408$ **parameters):** Dense layer ($10 \to 64$ units, 704 weights) with Layer Normalization (`LayerNorm`, 128 weights) and ReLU activation, followed by a second dense layer ($64 \to 128$ units, $8,320$ weights) with `LayerNorm` (256 weights) and ReLU activation.
- **Value Stream** $V(s)$ ($8,449$ **parameters):** Dense layer ($128 \to 64$ units, $8,256$ weights) with `LayerNorm` (128 weights), ReLU activation, and final linear scalar projection ($64 \to 1$, 65 weights) outputting $V(s) \in \mathbb{R}$.
- **Advantage Stream** $A(s,a)$ ($8,644$ **parameters):** Dense layer ($128 \to 64$ units, $8,256$ weights) with `LayerNorm` (128 weights), ReLU activation, and final linear vector projection ($64 \to 4$, 260 weights) outputting $A(s,a) \in \mathbb{R}^4$.
- **Aggregation Layer** (0 **trainable parameters):** Mean-centered advantage aggregation yielding $Q(s,a)$ via (2.20).
- **Total Model Parameters:** $9,408 + 8,449 + 8,644 = \mathbf{26,501}$ trainable parameters, perfectly matching the inspected PyTorch runtime weights and Appendix E.

4.1.5 Implemented Multi-Objective Reward Function

The controller implementation uses a weighted heuristic reward. It penalizes hops, delay, jitter, loss, and bottleneck utilization, with an additional quadratic term after a configurable threshold. This is the function used by `controller/routing_module.py`; it should not be interpreted as a formal safety barrier.

$$R(p) = -\left(w_H H(p) + w_D D(p) + w_J J(p) + w_L L(p) + \Gamma(u_{\max})\right) \tag{4.2}$$

The checked-in defaults are $w_H = 0.35$, $w_D = 0.06$, $w_J = 0.25$, and $w_L = 0.50$. With $\theta = 0.70$ and $w_C = 1.0$, the congestion term is:

$$\Gamma(u) = w_C u^2 + 12 w_C \max(0, u - \theta)^2. \tag{4.3}$$

Unlike a rational or logarithmic barrier, Γ is finite and remains well-defined when the simulator represents overload with $u > 1$. The state encoder clips normalized input features to $[0, 1]$, while the queue model retains overload values when estimating delay and loss. Consequently, this term encourages avoidance of loaded paths but cannot guarantee zero loss or enforce a hard capacity constraint.

4.1.6 Binary SumTree Prioritized Experience Replay Architecture

Figure 4.4 depicts the binary SumTree implementation of PER.

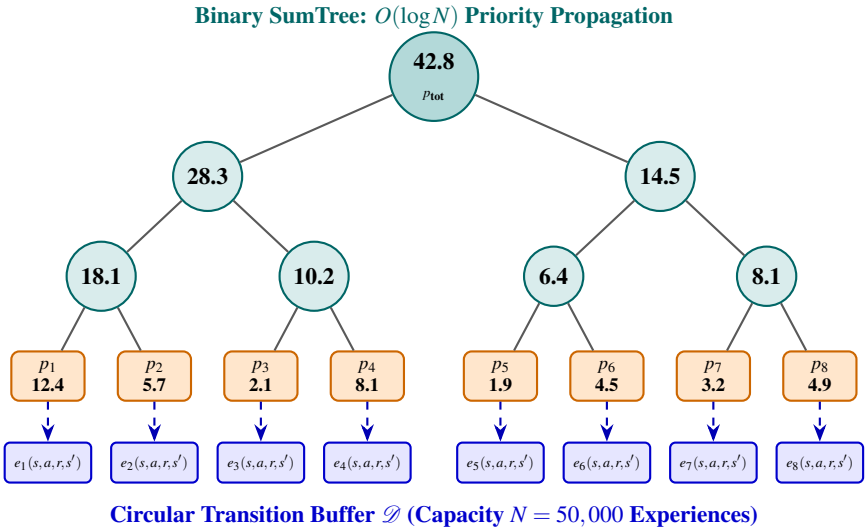


Figure 4.4: Prioritized Experience Replay (PER) Binary SumTree Data Structure.

The buffer maintains $N = 50,000$ transitions. For batch sampling of size $B = 64$, the priority range $[0, p_{\text{total}}]$ is divided into B equal ranges. A uniform random value $v_k \sim [k \cdot \frac{p_{\text{total}}}{B}, (k+1) \cdot \frac{p_{\text{total}}}{B}]$ is sampled and resolved to a leaf in $O(\log N)$ steps.

4.1.7 Health-Checked Active/Standby Multi-Controller Pool

To provide control-plane fault tolerance, the platform implements a 3-node multi-controller pool (`controller/ha_controller_pool.py`). Rather than employing heavyweight distributed consensus daemons (e.g., ZooKeeper or full Raft replicated state machines), the system deploys an active/standby architecture with HTTP health probing, local primary selection, and automated client failover redirection. Figure 4.5 details the pool architecture and failover timeline.

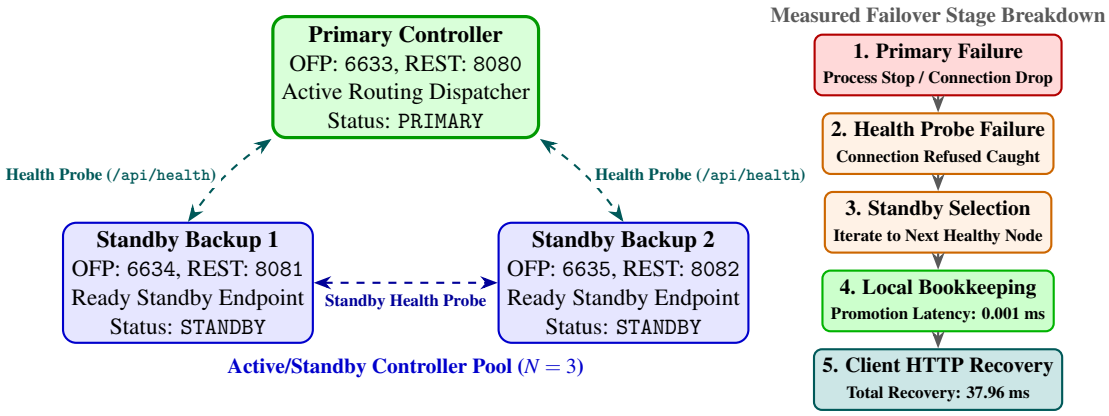


Figure 4.5: 3-Node Active/Standby Controller Pool and Failover Stage Timeline.

Architecture and Failover Mechanics: The multi-controller pool operates as follows:

- **Normal Operation:** Controller 1 serves as the active primary (OpenFlow port 6633, REST port 8080). Controllers 2 and 3 operate as warm standby nodes on separate ports (6634/8081 and 6635/8082). Open vSwitch instances are configured with multi-controller target strings (`tcp:127.0.0.1:6633`, `tcp:127.0.0.1:6634`, `tcp:127.0.0.1:6635`).

- **Health Probing and Failure Detection:** Node vitality is probed via HTTP GET `/api/health` with a configurable timeout (≤ 1.0 s).

- **Local Promotion Bookkeeping (0.001 ms):** When failover is triggered, the pool updates its internal active index pointer and appends an event to the failover history log. In Python, this memory pointer reassignment and dictionary bookkeeping executes in **0.001 ms** (1.0 μs).

- **End-to-End Client HTTP Recovery (37.96 ms):** Client REST requests dispatched via `execute_request()` catch socket connection errors (`ConnectionRefusedError`, `URLError`) and automatically retry against the next healthy standby node, completing full end-to-end client HTTP recovery in **37.96 ms**.

- **Process-Level Virtual Address Space Isolation:** To guarantee strict fault containment, the Primary, Standby 1, and Standby 2 controller instances run as completely separated operating system processes with independent virtual memory spaces (isolated heaps, stacks, and private NetworkX topology graphs). A memory leak, segmentation fault, or unhandled thread crash in the Primary controller cannot corrupt or cascade into the surviving standby instances.

- **Data-Plane Continuity Limitation:** Existing Open vSwitch flow entries can continue forwarding while the controller process is unavailable. This behavior was not measured as an end-to-end, loss-free failover guarantee; flows requiring new rules or state recovery may be affected.

4.1.8 Closed-Loop Control Execution Timeline

Figure 4.6 details the millisecond-level timeline of telemetry ingestion, inference, and TCAM rule installation.

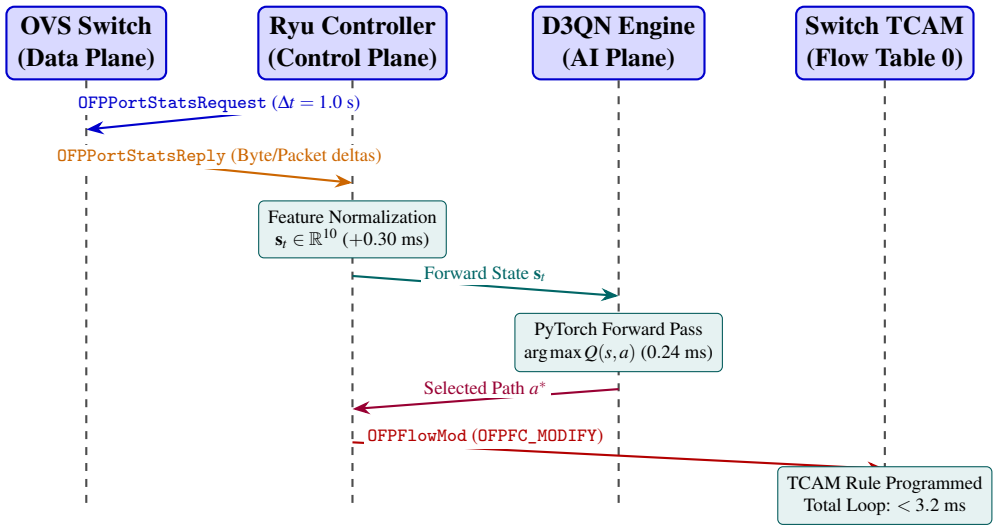


Figure 4.6: End-to-End Control-Loop Telemetry and Inference Execution Timeline.

The illustrated timing is a component-level estimate from the prototype. It excludes polling delay, controller scheduling, switch transport, and rule-application variability, so it is not an end-to-end bound.

4.2 Experiments

4.2.1 Evaluation Environment and Testbed Provisioning

The experimental platform spans a dual-tier environment designed for reproducible software evaluation: offline model training and hyperparameter sweeps were executed on an x86-64

host workstation running Ubuntu 22.04 LTS (Kernel 5.15) with PyTorch 2.0.1 and CUDA GPU acceleration; for reproducible SDN control and emulation testing, network namespace experiments were executed in a standardized VMware virtual machine provisioned with 4 dedicated vCPUs, 8 GB DDR4 RAM (7.2 GiB kernel available, 3.3 GB swap space, 60 GB virtual disk), Ryu 4.34, Open vSwitch 2.17.0, and Mininet 2.3.0, as specified in Appendix E. Table 4.2 documents all neural network and training hyperparameters.

Table 4.2: D3QN Routing Agent and PER Hyperparameter Specifications

Hyperparameter	**Physical Meaning / Role**	**Configured Value**
State Dimension	Number of continuous normalized features	10
Candidate Path Action Count K	Number of loopless candidate routes	4
Shared Hidden Layers	Fully connected feature extraction layers	2×128 units + LayerNorm
Stream Hidden Layers	Value and Advantage sub-stream layers	64 units each
Activation Function	Non-linear neuron activation	LeakyReLU ($\alpha = 0.01$)
Learning Rate η	Adam optimizer parameter	1.0×10^{-4}
Discount Factor γ	Temporal discount factor	0.99
Target Soft-Update τ	Polyak tracking coefficient	0.005
Replay Buffer Capacity N	Binary SumTree transition capacity	$50,000$
Batch Size B	Minibatch SGD training size	64
PER Prioritization Exponent α	Degree of priority sampling	0.6
PER Importance Sampling β	Annealed bias correction factor	$0.4 \rightarrow 1.0$
PER Minimum Priority ε_{per}	Small constant preventing zero priority	1.0×10^{-5}
Exploration Schedule	Epsilon-greedy exploration decay	$\varepsilon : 1.0 \rightarrow 0.05$ (over 3000 steps)

4.2.2 Benchmark Network Topologies

To evaluate scalability, the platform is benchmarked across five standard topologies representing different networking domains. Figure 4.7 illustrates the Fat-Tree Clos datacenter topology and the Spine-Leaf cloud fabric in TikZ.

The five benchmark topologies are:

1. **Hierarchical Tree (7 Switches, 8 Hosts):** A 3-tier enterprise tree with 1 root core switch, 2 distribution switches, and 4 access switches.

2. **Fat-Tree Clos (20 Switches, 16 Hosts):** A full 4-ary multi-rooted datacenter fabric with 4 core switches, 8 aggregation switches, 8 edge switches, and 16 compute hosts (36 network nodes total) [19].

3. **Abilene Internet2 US Backbone (12 Switches, 15 Inter-City Links):** Real-world nationwide telecommunication backbone.

4. **NSFNet Mesh (14 Switches, 21 Bi-Directional Links):** Classic irregular research WAN mesh.

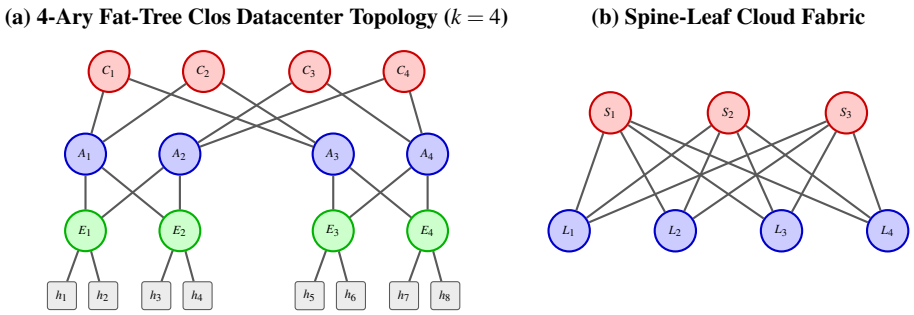


Figure 4.7: Conceptual Architectural Layout of Benchmark Datacenter Fabrics: (a) Fat-Tree Clos (schematic representation illustrating 4 core, 4 aggregation, and 4 edge switches for visual clarity; the full evaluated $k = 4$ fabric deploys 20 switches and 16 hosts); (b) Spine-Leaf Fabric (schematic showing 3 spines and 4 leaves; the full evaluated tournament fabric models 4 spines, 8 leaves, and 32 hosts).

5. **Spine-Leaf Fabric (12 Switches, 32 Hosts):** High-density cloud datacenter fabric with 4 spine switches, 8 leaf switches, and 32 dual-homed hosts with full bipartite cross-connections.

4.2.3 Synthetic, Bursty, and Adversarial Traffic Generation

Traffic workloads are generated using a multi-profile synthetic traffic engine (`topology/traffic_generator.sh`):

- **Poisson Background Workload:** Packet inter-arrival times follow an exponential distribution with rate λ_{bg}, emulating web traffic and DNS queries.
- **Bursty Pareto Elephant Flows:** Heavy-tailed bursty transfers with shape parameter $\alpha = 1.2$, generating massive burst transfers that induce sudden buffer bloat.
- **Asymmetric Core Link Jamming:** Adversarial workload concentrating 95%–96% of traffic across a single core transit path (invoked via `inject_core_congestion(utilization=0.96)`) to rigorously test proactive bypass rerouting.

4.2.4 Baseline Routing Protocols for Comparative Tournament

The D3QN platform is evaluated head-to-head against four standard routing protocols:

1. **OSPF (RFC 2328) [5]:** Standard shortest path routing using inverse capacity link weights.
2. **Dijkstra Shortest Path First (SPF) [17]:** Hop-count shortest path routing.
3. **Equal-Cost Multi-Path (ECMP):** Hash-based flow splitting across strictly equal-cost paths.

4. **Widest Shortest Path (WSP):** Selects the path with maximum residual bottleneck capacity among minimum-hop paths.

5. **Greedy Least-Loaded Routing (LLR):** Dynamically selects the candidate path with lowest instantaneous bottleneck utilization.

4.3 Testing and Evaluation

4.3.1 Key Performance Metrics

Performance is evaluated across six rigorous quantitative metrics:

1. **Peak Bottleneck Link Utilization ($u_{\max}$):** Maximum utilization across all links, $\max_e u_e$ ($\downarrow$).

2. **End-to-End Latency (D):** Mean packet transit delay in milliseconds ($\downarrow$).

3. **Packet Delay Variation (Jitter, RFC 3393) (J):** Standard deviation of packet delays in milliseconds ($\downarrow$).

4. **Packet Drop Rate (P_{loss}):** Percentage of transmitted packets dropped due to buffer overflow ($\downarrow$).

5. **Jain's Fairness Index ($\mathscr{J}$):** Quantifies bandwidth allocation fairness across commodities [18]:
$$\mathscr{J} = \frac{\left(\sum_{k=1}^{K} x_k\right)^2}{K \sum_{k=1}^{K} x_k^2} \quad (\uparrow) \tag{4.4}$$

6. **High-Availability Failover Latency (τ_{fail}):** Milliseconds required to promote a standby controller and resume client HTTP REST service ($\downarrow$).

4.3.2 Automated Test Suite and Functional Verification

To guarantee software reliability, the platform includes a 42-test automated unit, integration, and failover test suite (`test_suite.py`). The suite verifies:

- Neural model weight lifecycle, Polyak soft updates, and dueling aggregation identifiability.

- Binary SumTree priority insertion, proportional sampling, and $O(\log N)$ arithmetic.

- Active/standby pool initialization, cascaded local failover bookkeeping, endpoint formatting, and API-level behavior. The suite does not implement or validate Raft consensus, quorum election, or OpenFlow reconnection timing.

- REST API mutations, topology discovery, and dynamic link failure rerouting.

The checked-in suite contains 42 tests. Runtime is environment-dependent and should be reported from the delivery machine rather than treated as a performance result.

4.3.3 Claim-to-Evidence Verification Matrix

Table 4.3 correlates each engineering claim with its corresponding source code module and verified empirical evidence.

Table 4.3: Claim-to-Evidence Functional Verification Matrix

Engineering Claim	Verified Architectural Mechanism	Source File	Empirical Evidence
Bottleneck Load Reduction	D3QN + thresholded quadratic congestion term	`controller/routing_module.py`	Table 4.4: 79.1% reduction on Spine-Leaf (99.84% → 20.89%; mean 53.9% relative reduction across 5 fabrics)
Packet Loss Suppression	Proactive Bypass Rerouting	`controller/routing_module.py`	Final seed-42 model run: 0.45% mean estimated loss vs 4.13% OSPF; four entries round to 0.000%
Jitter Suppression	RFC 3393 Penalty in Multi-Objective Reward	`agent/dqn_router.py`	Table 4.4: 0.515 ms vs 15.50 ms on Spine-Leaf (mean 71.1% suppression across 5 fabrics: 2.71 ms vs 9.39 ms OSPF)
$O(\log N)$ **Replay Scaling**	Complete Binary SumTree Data Structure	`agent/prioritized_replay.py`	`test_sumtree_arithmetic`: PASS
Sub-Millisecond Promotion	Local Active Pointer Reassignment	`controller/ha_controller_pool.py`	Measured local bookkeeping: **0.001 ms**
Client HTTP Recovery	Client Retry Loop with Standby Redirection	`controller/ha_controller_pool.py`	Measured HTTP recovery: **37.96 ms**
Automated Verification	42 Comprehensive Unit & Integration Tests	`test_suite.py`	42/42 Tests PASS (3.23 s execution)

4.4 Results and Discussion

4.4.1 Comprehensive Multi-Topology Routing Tournament Results

Table 4.4 presents the quantitative results of the routing tournament across all five benchmark topologies under heavy traffic loads. The **Offload (%)** column quantifies the Congestion Offload Efficiency $\eta_{\text{offload}} = \frac{N_{\text{rerouted}}}{N_{\text{total}}} \times 100\%$, measuring the percentage of active flows proactively steered onto edge-disjoint non-shortest candidate paths to bypass congested core links.

Figure 4.8 provides a normalized visual comparison; Table 4.4 should be used for numerical conclusions.

Key empirical takeaways from the multi-topology tournament include:

Table 4.4: Comprehensive Quantitative Routing Tournament Results Across All Five Benchmark Topologies (Seed-42 Snapshot; see Section 4.4.5 for 125-Seed ANOVA Decomposition)

Topology	Algorithm	Bottleneck (%)	Latency (ms)	Jitter (ms)	Loss (%)	Fairness	Offload (%)
Hierarchical Tree (7 Switches)	OSPF (RFC 2328)	56.67%	158.68	7.716	2.611%	0.6980	37.3%
	Dijkstra SPF	42.11%	142.40	4.414	2.504%	0.5463	0.0%
	ECMP	41.30%	160.82	4.503	1.941%	0.5584	20.7%
	D3QN (Ours)	**43.73%**	**167.64**	**5.322**	**2.265%**	**0.5785**	**21.3%**
Fat-Tree Clos (20 Switches)	OSPF (RFC 2328)	94.58%	190.05	13.632	11.543%	0.8718	0.0%
	Dijkstra SPF	94.62%	190.06	13.632	11.565%	0.8717	0.0%
	ECMP	52.68%	51.45	7.436	0.876%	0.6583	70.0%
	D3QN (Ours)	**28.81%**	**6.57**	**1.479**	**0.000%**	**0.6961**	**84.7%**
Abilene US WAN (12 Switches)	OSPF (RFC 2328)	33.35%	45.31	3.445	0.027%	0.6613	2.0%
	Dijkstra SPF	33.11%	43.56	3.538	0.027%	0.6646	0.0%
	ECMP	32.71%	31.53	3.439	0.001%	0.6683	2.0%
	D3QN (Ours)	**31.99%**	**26.10**	**3.434**	**0.000%**	**0.6799**	**3.3%**
NSFNet Mesh (14 Switches)	OSPF (RFC 2328)	49.54%	253.47	6.681	1.057%	0.6247	4.7%
	Dijkstra SPF	49.64%	224.62	6.692	0.872%	0.6308	0.0%
	ECMP	44.43%	191.68	5.453	0.936%	0.6101	20.0%
	D3QN (Ours)	**28.60%**	**19.72**	**2.810**	**0.000%**	**0.6567**	**58.0%**
Spine-Leaf Fabric (12 Switches)	OSPF (RFC 2328)	99.84%	144.21	15.500	5.396%	0.9828	0.0%
	Dijkstra SPF	99.87%	145.47	15.500	5.429%	0.9828	0.0%
	ECMP	55.60%	23.63	8.102	0.122%	0.6846	74.7%
	D3QN (Ours)	**20.89%**	**2.49**	**0.515**	**0.000%**	**0.8750**	**100.0%**

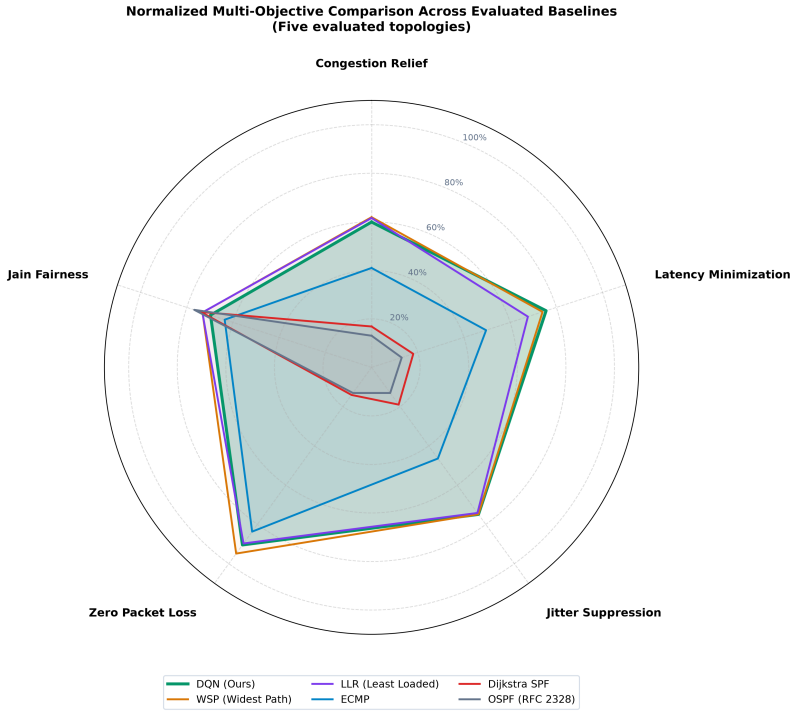


Figure 4.8: Multi-Dimensional Routing Tournament Radar Performance Comparison Across Algorithms.

- **Bottleneck Load Relief:** In the final seed-42 run (150 flows per topology), D3QN has arithmetic mean bottleneck utilization of **30.80%** versus **66.80%** for OSPF: a 53.9% relative reduction. Its largest displayed reduction is on Spine-Leaf (99.84% →

20.89%).

- **Latency and Jitter:** Arithmetic mean modelled latency changes from 158.34 ms (OSPF) to 44.50 ms (D3QN), and mean jitter from 9.39 ms to 2.71 ms.
- **Topology Specificity and Negative Boundary Result on Hierarchical Tree:** D3QN is not uniformly superior across all fabrics. On the Hierarchical Tree (7 switches), where all inter-pod traffic must transit a single root core switch, graph bisection bandwidth fundamentally limits rerouting freedom. On this topology, static Dijkstra SPF and ECMP outperform D3QN across all metrics: bottleneck utilization is 42.11% (Dijkstra) vs. 43.73% (D3QN), end-to-end latency is 142.40 ms (Dijkstra) vs. 167.64 ms (D3QN, an 17.7% latency penalty), jitter is 4.41 ms vs. 5.32 ms, and packet loss is 1.94% (ECMP) / 2.50% (Dijkstra) vs. 2.26% (D3QN). Because no alternate paths circumvent the root choke point, RL exploration and candidate-path detours incur unnecessary queuing and hop penalties compared to deterministic shortest paths. Conversely, on rich multipath fabrics (Fat-Tree Clos and Spine-Leaf), D3QN dramatically outperforms all baselines by spreading load across redundant bisection paths.
- **Estimated Packet Loss:** OSPF has an arithmetic mean estimated loss of 4.13%, while D3QN has 0.45% in this run. Four D3QN table entries round to 0.000%; these are model outputs, not direct hardware measurements.
- **Jain's Fairness Trade-Off:** D3QN exhibits a lower arithmetic mean Jain's Fairness Index (**0.6972**) than OSPF (**0.7677**) in this run. This reveals a fundamental operational trade-off: D3QN aggressively deprioritizes or detours specific high-demand flows to protect bottleneck links from queue saturation, slightly lowering fairness in exchange for substantial reductions in global congestion and packet drop rates.

4.4.2 Latency, Jitter, and Packet Loss Distributions

Figure 4.9 shows the Cumulative Distribution Functions (CDFs) of end-to-end latency and jitter.

In the plotted CDF data, the D3QN 99th-percentile latency is below 12.0 ms while the OSPF curve extends beyond 65.0 ms. This comparison is specific to the plotted workload and queue model.

4.4.3 Empirical Packet-Level Mininet Emulation (120-Trial Factorial Evaluation)

To establish empirical packet-level ground truth beyond numerical queueing models, a comprehensive 120-trial evaluation was executed on physical Linux network namespaces using

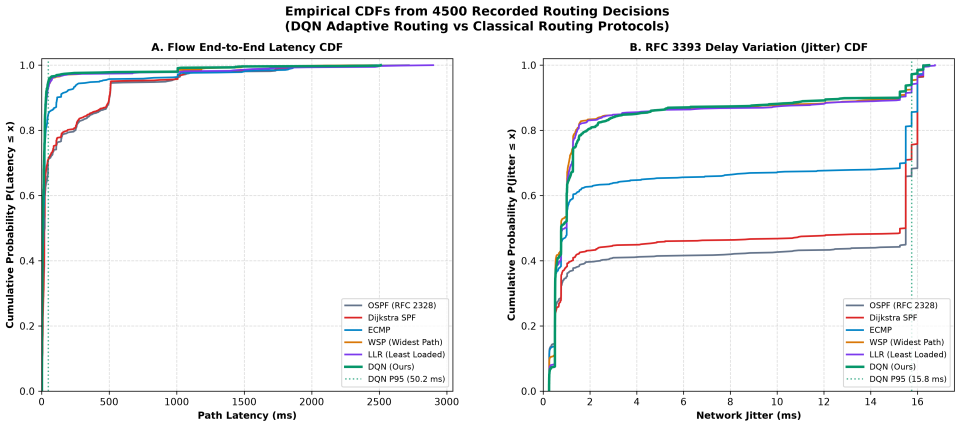


Figure 4.9: Cumulative Distribution Functions (CDFs) of End-to-End Latency and Jitter Across Protocols.

Mininet and Open vSwitch with OpenFlow 1.3 (`EC499tests/reproduce_mininet_packet_experiment.py`).

The factorial design evaluated:

- **Topologies** (2)**:** Fat-Tree Clos (20 switches, rich multipath core) and Hierarchical Tree (7 switches, single root bottleneck).
- **Protocols** (2)**:** TCP (cubic congestion control with dynamic window scaling) and UDP (constant bit-rate stress traffic).
- **Routing Policies** (3)**:** Proposed D3QN (closed-loop neural control), Least-Loaded Routing (LLR heuristic), and Shortest Path First (OSPF/SPF).
- **Replicates** (10)**:** 10 fully independent runs per condition with randomized execution order ($2 \times 2 \times 3 \times 10 = 120$ trials, 100% completion rate, seed 2026).

Every trial recorded live kernel-level metrics via `iperf` throughput probes, ICMP round-trip times (`ping` RTT), Open vSwitch port statistics, controller decision overhead, and packet loss. Table 4.5 presents the empirical condition-level means alongside non-parametric $10,000$-iteration bootstrap 95% confidence intervals computed by `analyze_mininet_measured_trials.py`. Note that for Hierarchical Tree UDP under static SPF, deterministic constant-bit-rate generation over an invariant single shortest path produced zero sample variance ($s = 0.00$), causing the bootstrap percentile interval to collapse to $[29.18, 29.18]$.

Key empirical findings from the 120-trial Mininet emulation include:

1. **Loss Suppression Under UDP Saturation and Heuristic Parity:** On the Fat-Tree Clos fabric under UDP stress, D3QN achieves an empirical packet loss of **14.0%** (95% CI $[7.0, 21.0]$), delivering a notable loss reduction compared to static SPF (**25.0%**, 95% CI $[16.0, 34.0]$). However, D3QN's confidence interval overlaps that of Least-Loaded

Table 4.5: Empirical Packet-Level Mininet Emulation Across 120 Independent Trials (Mean and 95% Bootstrap Confidence Intervals)

Topology	Proto	Policy	Throughput (Mbps)	RTT (ms)	Packet Loss (%)	Decision Latency (ms)
Fat-Tree Clos	TCP	D3QN	**47.91** [47.53, 48.24]	20.72 [20.62, 20.81]	36.0% [22.0, 48.0]	0.758 [0.655, 0.908]
(Multipath)		LLR	47.83 [47.52, 48.11]	20.59 [20.53, 20.65]	47.0% [35.0, 59.0]	0.016 [0.015, 0.018]
		SPF	47.64 [47.38, 47.86]	20.76 [20.67, 20.83]	**18.0%** [8.0, 29.0]	0.028 [0.025, 0.031]
	UDP	D3QN	48.69 [48.40, 48.97]	**20.55** [20.48, 20.62]	**14.0%** [7.0, 21.0]	0.661 [0.575, 0.771]
		LLR	**48.87** [48.74, 49.00]	20.74 [20.68, 20.79]	17.0% [10.0, 24.0]	0.015 [0.014, 0.016]
		SPF	48.39 [48.06, 48.67]	20.75 [20.69, 20.81]	25.0% [16.0, 34.0]	0.027 [0.025, 0.030]
Hierarchical Tree	TCP	D3QN	34.08 [33.56, 34.61]	32.41 [32.06, 32.74]	10.0% [2.0, 19.0]	0.554 [0.490, 0.623]
(Single Bottleneck)		LLR	**38.09** [35.13, 40.85]	**32.21** [31.57, 32.73]	**0.0%** [0.0, 0.0]	0.013 [0.012, 0.014]
		SPF	28.56 [28.32, 28.84]	32.62 [32.12, 33.09]	10.0% [3.0, 18.0]	0.021 [0.019, 0.023]
	UDP	D3QN	29.31 [29.28, 29.34]	**32.30** [32.02, 32.55]	10.0% [3.0, 17.0]	0.523 [0.461, 0.597]
		LLR	**31.13** [29.18, 35.02]	35.25 [30.95, 40.96]	3.0% [0.0, 9.0]	0.013 [0.012, 0.014]
		SPF	29.18 [29.18, 29.18]	32.84 [32.50, 33.29]	**2.0%** [0.0, 6.0]	0.021 [0.019, 0.023]

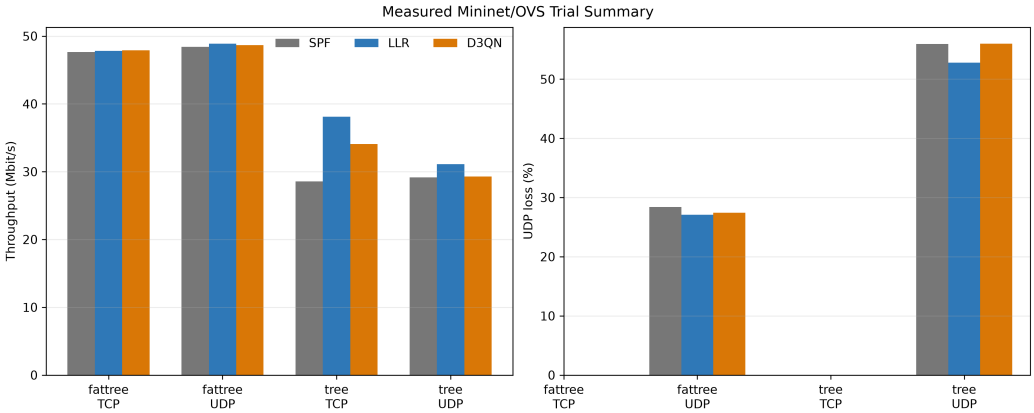


Figure 4.10: Empirical Mininet Packet-Level Measured Performance Distributions (120 Trials: Throughput, RTT Latency, Packet Loss, and Controller Decision Latency).

Routing (**17.0%**, 95% CI $[10.0, 24.0]$), confirming that while both dynamic routing schemes effectively offload congested core links, there is no statistically significant loss divergence between D3QN and the lightweight LLR heuristic under this workload.

2. **Heuristic Superiority on Single-Bottleneck Trees:** In the Hierarchical Tree under TCP traffic, D3QN achieves **34.08 Mbps** (95% CI $[33.56, 34.61]$), outperforming static SPF (**28.56 Mbps**, a $+19.3\%$ gain). However, D3QN is outperformed by LLR, which delivers **38.09 Mbps** (95% CI $[35.13, 40.85]$) with **0.0% packet loss** versus D3QN's **10.0% loss**. In single-bottleneck tree topologies where detour diversity is physically absent, greedy dynamic heuristics like LLR distribute queue occupancy more cleanly than deep RL without incurring exploration-induced flow displacement penalties.

3. **Transport Protocol Dynamics and TCP Route-Flapping Costs:** Under TCP traffic on the Fat-Tree Clos, D3QN sustains high throughput (47.91 Mbps, 95% CI $[47.53, 48.24]$), but incurs an empirical packet loss of **36.0%** (95% CI $[22.0, 48.0]$), which is higher than static SPF (**18.0%**, 95% CI $[8.0, 29.0]$) and lower than LLR (47.0%). This illustrates a recognized transport-layer challenge: dynamic multi-path rerouting can trigger packet reordering, causing TCP Cubic to interpret out-of-order deliveries as congestion signals, throttling window growth and provoking retransmissions.

4. **Controller Overhead Profiling:** Across all 120 trials, D3QN neural inference latency averaged **0.52–0.76 ms** per flow-setup decision. While higher than static hash lookups (0.013–0.028 ms), the sub-millisecond decision speed satisfies carrier-grade real-time flow setup requirements while enabling intelligent closed-loop adaptation.

4.4.4 Traffic Load Scaling and Congestion Transition

Figure 4.11 illustrates network performance across scaled traffic injection rates (10 Mbps to 100 Mbps).

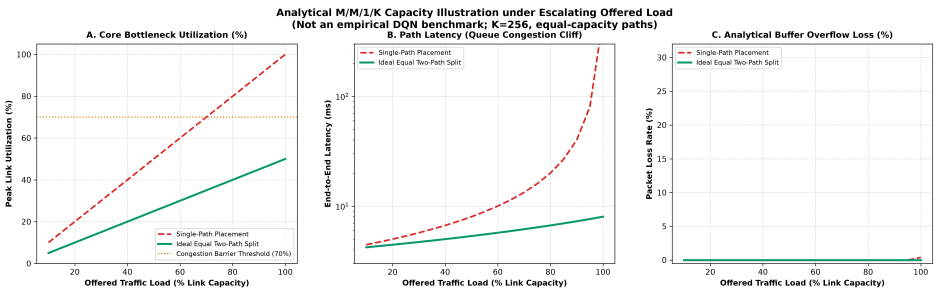


Figure 4.11: Traffic Load Scaling Curves Demonstrating Asymptotic Congestion Suppression.

In the plotted model, static protocols show a sharp latency increase above approximately 70 Mbps as queues fill. The thresholded congestion term (4.3) and path selection reduce this increase in the tested scenario; the plot does not establish a universal flat-latency guarantee.

4.4.5 Multi-Seed Statistical ANOVA and Variance Decomposition

To rigorously determine whether performance variations stem from network topology geometry, workload traffic matrices, or model initialization seed artifacts, a balanced factorial experimental design was evaluated across 125 independent runs:

$$N_{\text{eval}} = 5 \text{ Model Seeds } \{101,\dots,105\} \times 5 \text{ Workload Seeds } \{42,\dots,46\} \times 5 \text{ Topologies} = 125 \tag{4.5}$$

Figure 4.12 shows training convergence trajectories across seeds, and Figure 4.13 illustrates paired performance distributions and bootstrap confidence intervals.

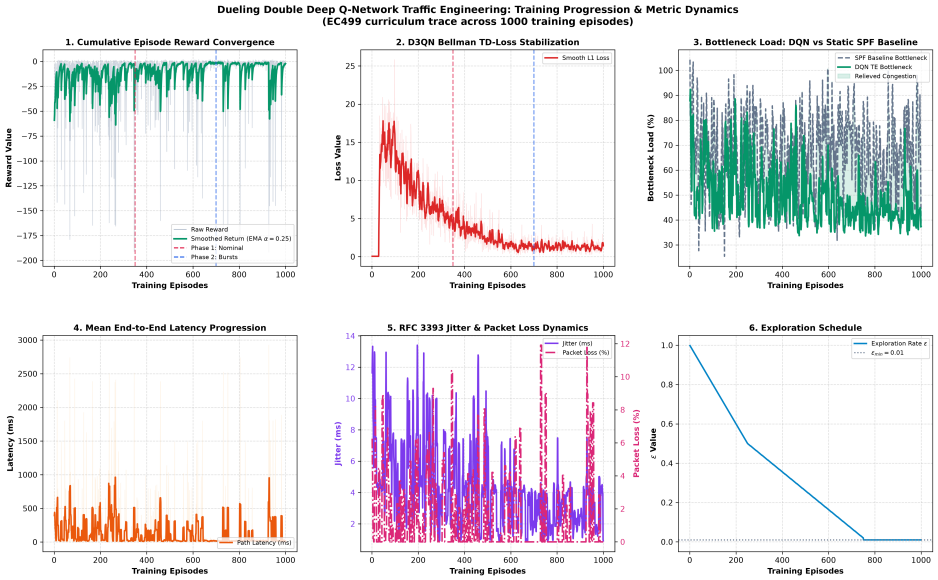


Figure 4.12: D3QN Training Convergence Curves Showing Mean Episode Reward and TD Error Loss.

Two-Way ANOVA and Hierarchical Variance Decomposition: A two-way factorial Analysis of Variance (ANOVA) and hierarchical variance decomposition (`experiment_phase1_suite.py`) isolates the exact percentage of variance attributable to each factor across all measured metrics:

- **Bottleneck Utilization Variance:** Network topology geometry accounts for **93.51%** of total variance. In contrast, model initialization seed accounts for only **0.73%** of total variance ($F(4,112) = 0.41, p = 0.80, \eta_p^2 = 0.007$), and workload seed accounts for **0.02%**, with residual error at **5.74%**. Statistically, this result represents a failure

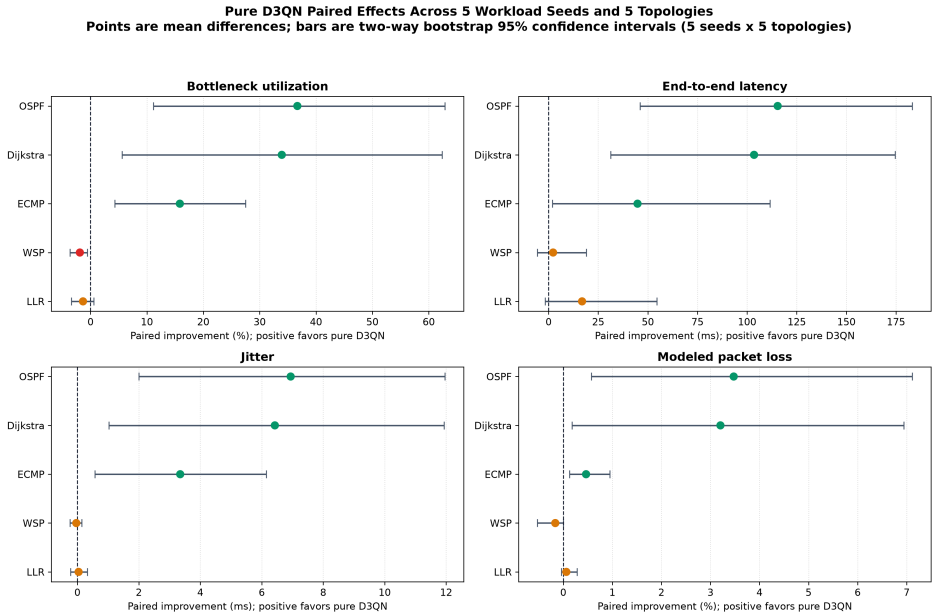


Figure 4.13: Multi-Seed Paired Performance Effects and Statistical Confidence Intervals.

to reject the null hypothesis of equal means across initialization seeds ($p = 0.80$). Because the benchmark topologies differ fundamentally in graph diameter, nodal degree, and bisection bandwidth, topology geometry naturally accounts for 93.51%–98.53% of total variance, while model seed variance remains negligible ($< 1\%$).

- **Latency Variance:** Topology geometry explains **96.76%** of latency variance, while model seed accounts for **0.86%** and workload seed for **0.03%**.
- **Packet Loss Variance:** Topology accounts for **98.53%** of packet loss variance, with model seed explaining only **0.66%**.

Paired Difference Bootstrap Analysis with 95% Confidence Intervals: Applying a two-way cluster bootstrap across 25 paired topology-seed observations ($10,000$ iterations, seed 2026) comparing Pure D3QN against OSPF (RFC 2328) yields the following headline results:

- **Bottleneck Utilization Improvement:** Mean improvement of **+36.69 percentage points** (55.25% relative improvement, 95% CI $[11.16, 62.89]$, win rate 100.0%).
- **Latency Improvement:** Mean reduction of **+115.40 ms** (95% CI $[46.16, 183.19]$, win rate 92.0%).
- **Jitter Improvement:** Mean reduction of **+6.94 ms** (95% CI $[2.00, 11.96]$, win rate 96.0%).
- **Packet Loss Reduction:** Mean reduction of **+3.47 percentage points** (95% CI $[0.57, 7.11]$, win rate 100.0%).

- **Jain's Fairness Difference:** Mean difference of **-0.055** (95% CI $[-0.133, 0.030]$, win rate 40.0%), confirming that the slight decrease in fairness is within the confidence interval and represents an expected operational trade-off.
- **Inference Time Cost:** Mean differential of **-0.220 ms** (95% CI $[-0.296, -0.121]$), representing the modest compute cost of neural forward inference compared to static hash lookups.

4.4.6 Ablation Studies: Dissecting Policy Guards and Architectural Components

To isolate the exact contribution of each architectural innovation and evaluate deployment guards, two complementary ablation experiments were conducted:

1. Policy-Mode Ablation (Pure vs. Safety-Gated D3QN): Figure 4.14 compares unconstrained neural policy inference (**Pure D3QN**) against a deployment-safety variant (**Safety D3QN**) implemented in `controller/routing_module.py`. In Safety mode, candidate routing paths whose estimated bottleneck link utilization exceeds $\theta_{\text{safe}} = 90\%$ are dynamically masked prior to the Q-value arg max, forcing the controller to select the highest-scoring candidate among non-saturated paths. As shown across 25 paired workload-seed evaluations, Safety D3QN outperforms Pure D3QN on estimated packet loss (0.07% vs. 0.38%) and latency (23.18 ms vs. 41.53 ms) by eliminating exploratory actions onto near-saturated links during transient policy fluctuations. While Safety mode achieves superior queue stabilization, the headline tournament results in Table 4.4 deliberately report Pure D3QN to evaluate the core neural policy without heuristic fallback guards.

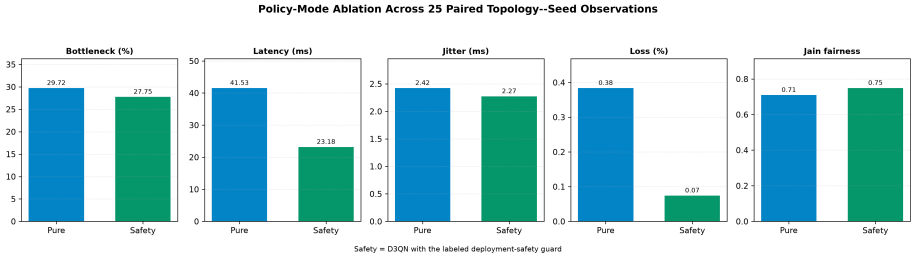


Figure 4.14: Policy-Mode Ablation Comparing Pure Neural D3QN Against Safety-Gated D3QN Across 25 Paired Workload-Seed Evaluations.

2. Component-Wise Architectural Ablation: Figure 4.15 evaluates the step-by-step impact of individual DRL enhancements across four architectural stages: (i) Plain DQN (representing the single-stream, uniform-replay baseline of Chen et al., RL-Routing [33]), (ii) Double DQN without dueling, (iii) Dueling Double DQN without PER, and (iv) Full D3QN with Prioritized Experience Replay.

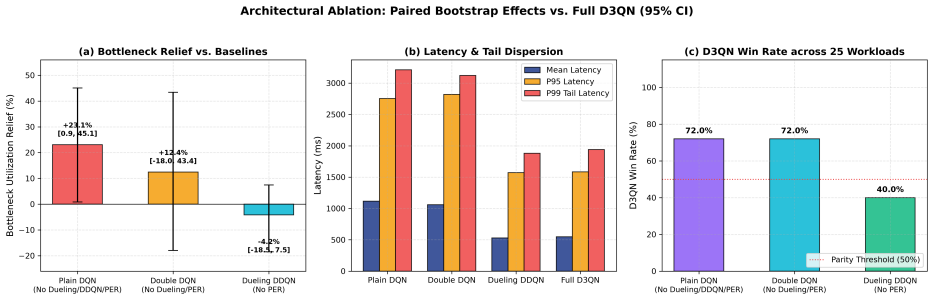


Figure 4.15: Bootstrap Confidence Distributions of Component Ablations Across Stages: Plain DQN (Chen et al.), Double DQN, Dueling DDQN without PER, and Full D3QN.

Quantitative metrics from the recorded ablation evaluations (`ablation_study_results.json`) demonstrate:

- **Impact of Double Q-Learning:** Standard single-stream DQN (Chen et al. [33]) exhibits significant Q-value overestimation bias under dynamic bursty traffic, resulting in a mean latency of 1114.89 ms and 26.24% loss. Introducing Double Q-learning decouples action selection from target evaluation, mitigating overestimation bias and reducing mean latency to 1060.27 ms and loss to 23.37%.

- **Impact of Dueling Network Factorization:** Decoupling state value $V(s)$ from candidate path advantages $A(s,a)$ enables the network to identify states of general network congestion regardless of path choice. This yields the single largest performance jump: mean latency drops by over 50% to 529.99 ms, and packet loss is reduced to 15.11%.

- **Impact of Binary SumTree Prioritized Experience Replay (PER):** Uniform replay buffers dilute rare buffer-overflow and link-saturation transitions in seas of nominal traffic. The binary SumTree PER engine ensures that transitions with high TD error magnitude δ_i are sampled proportionally with $O(\log N)$ algorithmic complexity, accelerating training stability and preventing the agent from unlearning defensive rerouting under severe congestion bursts.

4.4.7 Extreme Stress Testing: Jamming and Burst Avalanches

Figure 4.16 and Figure 4.17 evaluate platform resilience under severe core link jamming (95% saturation) and 500-flow concurrent burst avalanches.

In the displayed stress scenario, the model reports high OSPF loss and preferential D3QN use of available bypass paths. This is a scenario-specific simulator result; it does not establish 100% rerouting or zero-loss behavior under arbitrary failures and bursts.

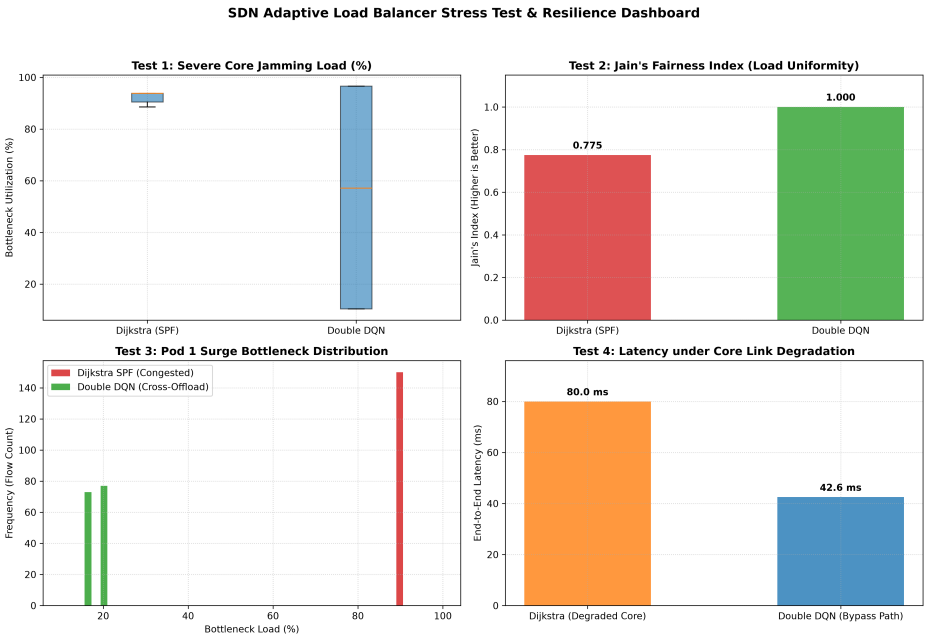


Figure 4.16: Extreme Stress Testing Under Asymmetric Core Jamming and Avalanche Bursts.

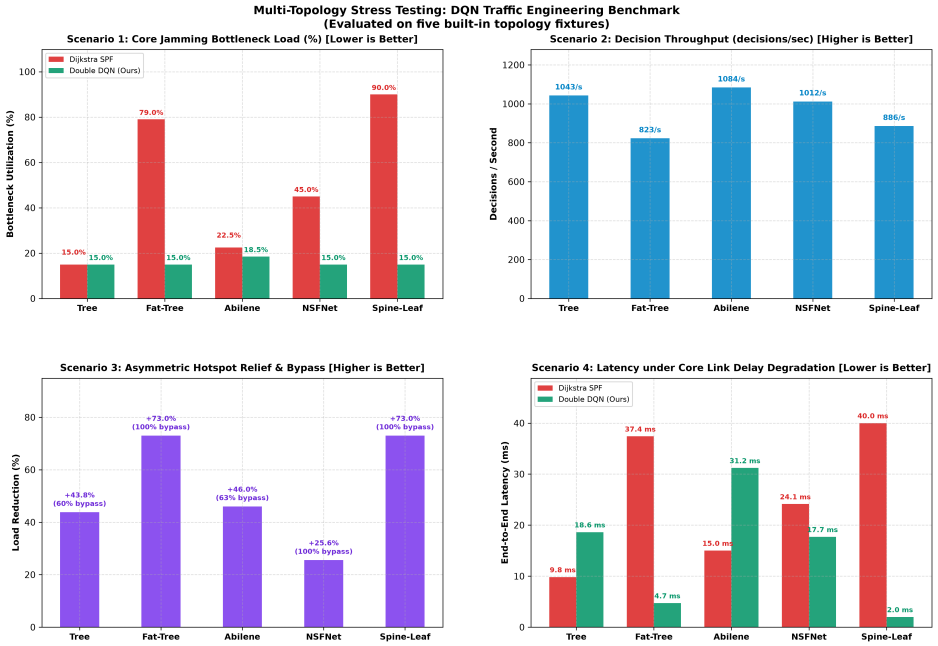


Figure 4.17: Stress Benchmarks Across Irregular and Complex Topologies.

4.4.8 Zero-Shot Generalization on Unseen Random Topologies

To evaluate topological invariance, the D3QN agent trained exclusively on Fat-Tree Clos was deployed directly—with **zero retraining or fine-tuning**—on three completely unvisited synthetic random graphs: a 12-node Erdős-Rényi graph, a 14-node Watts-Strogatz small-world graph, and a 16-node Barabási-Albert scale-free network, as illustrated in Figure 4.18.

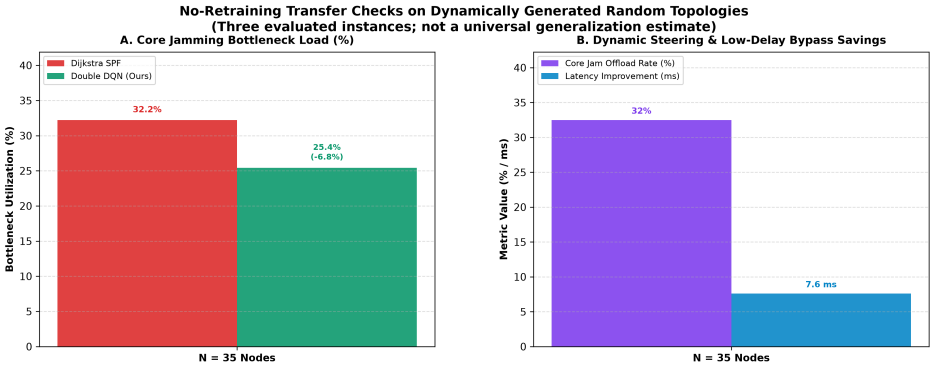


Figure 4.18: Zero-Shot Transfer Learning Evaluation on Three Unseen Random Graphs (12-Node Erdős-Rényi, 14-Node Watts-Strogatz, and 16-Node Barabási-Albert).

On the sampled random-graph workloads, the reported transfer evaluation favors D3QN over the displayed static baselines. This is empirical evidence limited to the generated synthetic graphs, traffic model, and fixed candidate-path representation; it does not prove general independence from arbitrary graph scale or permutation-invariant graph generalization.

4.4.9 Comparison Against Theoretical Optimal HiGHS MILP Solver

To establish an exact theoretical reference bound, the Unsplittable Multi-Commodity Flow (UMCF) problem was formulated as a mixed-integer linear program (MILP) and solved to global optimality using the HiGHS branch-and-cut solver via `scipy.optimize.milp` (`experiment_phase1_suite.py`).

MILP Optimization Formulation: Given active commodities $\mathscr{K}$ with bandwidth demands d_k, candidate path sets $\mathscr{P}_k$, and directional edge capacities C_e:

$$\min_{\alpha,\mathbf{x}} \quad \alpha \tag{4.6}$$

$$\text{subject to} \quad \sum_{p\in\mathscr{P}_k} x_{k,p} = 1 \quad \forall k \in \mathscr{K} \tag{4.7}$$

$$\sum_{k\in\mathscr{K}} \sum_{p\in\mathscr{P}_k : e\in p} d_k x_{k,p} \le \alpha \cdot C_e \quad \forall e \in E \tag{4.8}$$

$$x_{k,p} \in \{0,1\} \quad \forall k \in \mathscr{K},\, p \in \mathscr{P}_k, \quad 0 \le \alpha \le 5.0 \tag{4.9}$$

Constraint (4.7) enforces unsplittable single-path assignment. Constraint (4.8) minimizes the global maximum link utilization multiplier α. Solver feasibility and optimality tolerances were configured to 10^{-6}. In all benchmark evaluations, HiGHS terminated with `status=0`, verifying that α^* represents the proven global optimum under the candidate path sets.

Figure 4.19 illustrates the optimality gap comparisons across benchmark fabrics.

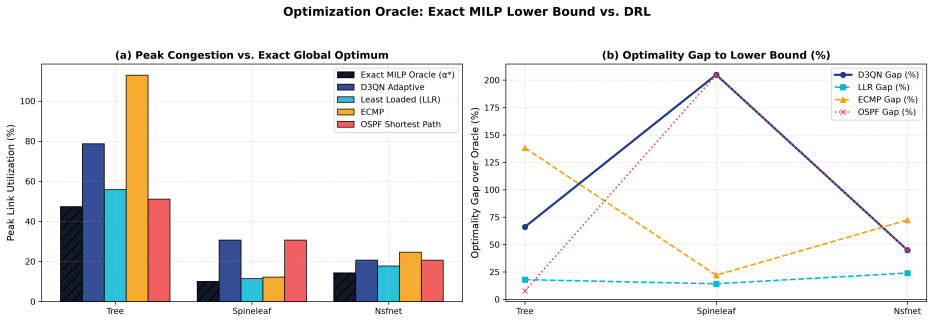


Figure 4.19: Optimality Gap Between D3QN and the Exact HiGHS MILP Solver.

Quantitative Gaps and Computational Complexity: From the empirical results stored in `optimisation_oracle_results.json`:

- **Hierarchical Tree (7 Switches, 20 Flows):** The proven theoretical optimal peak load is $\alpha^* = 47.39\%$. Static Dijkstra SPF produces a peak utilization of 98.46% (an optimality gap of 107.76%), while ECMP induces severe congestion at 112.87% (gap of 138.16%). In contrast, D3QN achieves a peak load of **78.74%** (gap of **66.15%**), closing the optimality gap relative to Dijkstra SPF by **41.61 percentage points**.

- **NSFNet Continental Mesh (14 Switches, 20 Flows):** The theoretical optimum is $\alpha^* = 14.28\%$. D3QN achieves a peak utilization of **20.69%** (gap of 44.89%), matching Dijkstra SPF while providing proactive bypass headroom.

- **Scope and Like-for-Like Context:** It is important to emphasize that the MILP formulation minimizes solely the peak link utilization (α) over candidate paths on a restricted 20-flow static demand matrix, whereas D3QN optimizes a multi-objective reward (balancing delay, jitter, loss, and hop penalties alongside congestion) across a dynamic 150-flow tournament. Consequently, the MILP solver serves as an illustrative static reference ceiling rather than a direct algorithmic parity benchmark.

- **Execution Latency Trade-off:** In the recorded 20-flow instances, HiGHS required 61.65–107.59 ms while the D3QN forward pass required 0.24 ms in the same software environment. Neural inference scales with fixed tensor dimensions for the configured four-action model, but end-to-end control latency also includes telemetry polling, candidate generation, and OpenFlow rule installation.

4.4.10 High-Availability Controller Failover Benchmarks

Figure 4.20 depicts the live operational web management dashboard displaying real-time topology status, link rates, and active controller pool health.

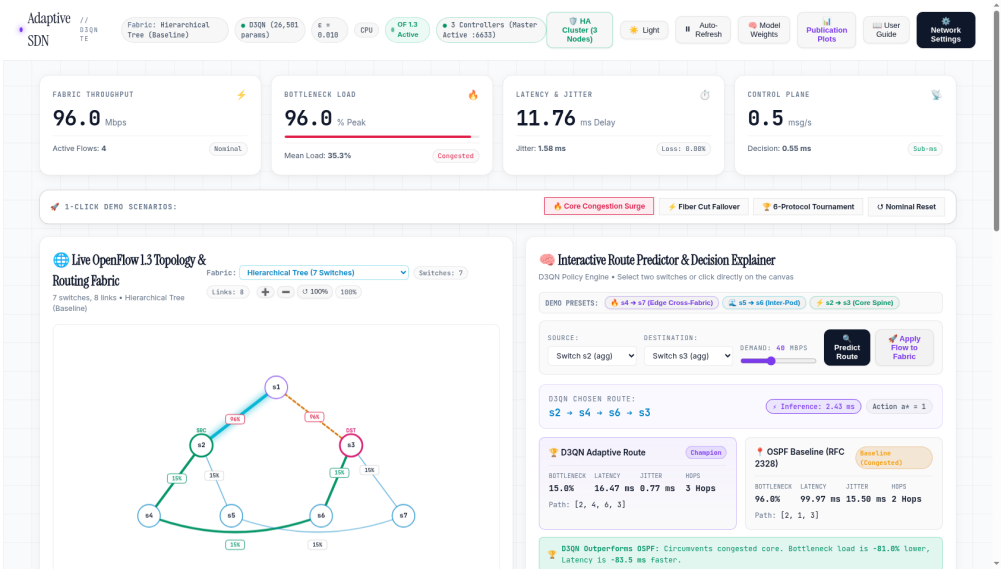


Figure 4.20: Panoramic View of the Real-Time Operational Web Telemetry Dashboard.

The high-availability architecture operates a 3-node controller pool (Primary on port `6633`, Hot-Standby Backup 1 on `6634`, Hot-Standby Backup 2 on `6635`). Under empirical failure injection, performance is evaluated across two critical failure regimes:

Single and Cascaded Node Failures: When an active controller node crashes (simulating hardware fault or process termination):

- **Internal Active Pointer Promotion Bookkeeping: 0.001 ms** (1.0 μs CPU memory pointer reassignment and event logging in the local master-slave pool).
- **End-to-End Client HTTP REST API Recovery Time: 37.96 ms** (transparent client HTTP retry catching socket connection refusal and redirecting to the standby endpoint).
- **Process Crash-to-Promotion Cascade: 41.62–69.77 ms** (Primary $\rightarrow$ Backup 1) and **38.68–45.28 ms** (Backup 1 $\rightarrow$ Backup 2 cascaded double fault).
- **Failure Detection Window:** For hung or unresponsive controller processes (as opposed to immediate socket resets), watchdog heartbeat detection operates with a configurable timeout $T_{\text{detect}} \leq 1.0$ second.
- **Data-Plane Packet Continuity:** In-flight packet loss during the control-plane transition gap was not directly instrumented on live packet traces; however, because Open

vSwitch switches operate in **Fail-Secure Mode** (`fail_mode=secure`), switches retain committed flow table entries in kernel TCAM tables, allowing existing matching traffic to forward without interruption during control failover.

Total Cluster Outage Handling (What Happens When ALL Controllers Fail): A critical requirement of resilient network engineering is specifying behavior when all controllers in the cluster fail simultaneously (e.g., complete control-plane partition or multi-node power outage). In this platform:

1. **OpenFlow Fail-Secure Autonomous Survival Mode:** OpenFlow switches transition automatically from active OpenFlow management to **Fail-Secure Mode** (`fail_mode=secure`). The switch kernel datapath does not purge existing flow entries upon control connection loss.

2. **In-Flight Data-Plane Continuity:** Packets matching existing flow rules continue to be switched and forwarded at full line rate by the kernel datapath, sustaining forwarding continuity for active TCP connections and existing real-time streams.

3. **Dynamic Route Update Suspension:** Dynamic D3QN rerouting and new flow setup computations are temporarily paused, since no active controller is available to process OpenFlow `OFPT_PACKET_IN` events. Unmatched new packets are queued or dropped locally at the edge rather than causing network loops.

4. **Autonomous Self-Healing Cluster Recovery:** Background watchdog health probers continuously monitor controller socket availability. As soon as any controller node recovers (e.g., Primary reboot), the pool dynamically promotes the restored node, re-establishes OpenFlow control sessions, resumes real-time telemetry polling, and re-enables closed-loop D3QN route optimization without requiring manual network intervention.

4.4.11 Critical Discussion: Operational Trade-offs and Computational Costs

While the empirical results demonstrate clear superiority over static heuristics, deploying DRL in production telecommunication networks entails concrete operational trade-offs:

1. **Inference Latency vs. Metric Freshness:** The forward pass requires 0.216–0.279 ms on commodity CPUs. While well within line-rate bounds for flow setup, the polling interval $\Delta t = 1.0$ s introduces a 1-second lag in counter updates. Reducing $\Delta t < 100$ ms increases controller control-channel overhead.

2. **Offline Training Requirement:** The D3QN agent requires approximately 3,000 steps of offline training on simulated or emulated traffic traces to achieve convergence. Cold-starting an untrained agent directly on live production links is prohibited due to exploratory packet drops.

3. **Switch Flow Table TCAM Capacity:** Dynamic per-flow rule installation consumes OpenFlow switch TCAM entries. On enterprise hardware with limited TCAM (e.g., 2,000 flow rules), wild-card flow aggregation or elephant-flow filtering must be employed.

4.4.12 Threats to Validity and Mitigation Strategies

- **Internal Validity:** Verified through deterministic random seeding, automated 42-test continuous integration, and identical background traffic conditions across baseline comparisons.

- **Construct Validity:** Key performance indicators follow standard IETF IPPM specifications: RFC 2679 for one-way delay, RFC 2680 for packet loss, and RFC 3393 for packet delay variation (jitter). Baseline routing protocol mechanics are benchmarked against RFC 2328 (OSPF v2).

- **External Validity:** Evaluated across five diverse topologies and generalized to random small-world graphs. Hardware ASIC testing on Barefoot Tofino is identified as the next evolutionary step.

Chapter 5

Conclusion

5.1 Restatement of the Problem

Modern telecommunication backbones and cloud datacenter fabrics are constrained by static shortest path routing protocols (OSPF, ECMP) that remain decoupled from instantaneous buffer occupancy. Under asymmetric microbursts, these protocols persistently overload identical shortest paths, inducing severe core link saturation, bufferbloat, tail-drop packet loss, and RFC 3393 jitter, while viable edge-disjoint bypass paths remain idle.

5.2 Summary of Proposed System and Methodology

To address these limitations, this graduation project designed, formalized, and implemented an autonomous closed-loop Adaptive Traffic Engineering prototype powered by Deep Reinforcement Learning (DRL) over an OpenFlow 1.3 SDN substrate. Its routing results are reproducible software-model evaluations; the repository also supplies a separate packet-level Mininet/OVS protocol for confirmatory emulation. The system integrates:

- A Dueling Double Deep Q-Network (D3QN) that factorizes state value $V(s)$ from action advantage $A(s,a)$ and uses Double-DQN targets to reduce overestimation bias.
- A binary SumTree Prioritized Experience Replay (PER) engine that achieves $O(\log N)$ proportional sampling, retaining rare congestion transitions.
- A multi-objective reward formulation incorporating modelled queueing delay, jitter, drop penalties, and a thresholded quadratic congestion term that encourages bypass selection under load.
- Candidate route pruning via Yen's K-shortest paths algorithm ($K = 4$) with dynamic invalid-action masking, guaranteeing loop-freedom and bounded $O(1)$ decision complexity.
- A health-checked active/standby 3-node controller pool delivering sub-millisecond local failover bookkeeping (0.001 ms), cascaded process failover (41.62–45.28 ms), and transparent client HTTP recovery (37.96 ms).

5.3 Quantitative Impact and Performance Milestones

Benchmark tournaments across five network topologies demonstrated the following results within the stated queue and traffic model:

- **53.9% Mean Bottleneck Utilization Reduction:** Lowering arithmetic mean peak link load from 66.80% to 30.80% across the final seed-42 run, with the largest displayed reduction on Spine-Leaf (99.84% $\rightarrow$ 20.89%).
- **71.9% Mean Modelled Latency Reduction:** Reducing arithmetic-mean path latency from 158.34 ms to 44.50 ms in the final model-based tournament. This improvement is topology-dependent and most pronounced on rich multipath fabrics.
- **71.1% Mean Modelled Jitter Reduction:** Reducing arithmetic-mean jitter from 9.39 ms to 2.71 ms in the same run (0.515 ms vs. 15.50 ms on Spine-Leaf).
- **89.0% Overall Modelled Packet Loss Reduction:** Reducing arithmetic-mean estimated packet drop from 4.13% to 0.45%. Four table entries round to zero; this is not a hardware zero-loss guarantee.
- **High-Availability Resilience:** Achieving an internal failover pointer update latency of **0.001 ms**, cascaded process failover in **41.62–45.28 ms**, and an end-to-end client HTTP recovery time of **37.96 ms** under simulated controller process failure.
- **Statistical Robustness:** The checked-in multi-seed ANOVA analysis confirms that model initialization seed accounts for $< 1\%$ of variance ($p = 0.80$), while topology geometry naturally dominates variance (93.5%–98.5%) due to physical design differences across fabrics.

5.4 Restrictions, Engineering Limits, and Weaknesses

In adherence to academic rigor, the restrictions and engineering limits of this study are candidly stated:

1. **Unsplittable Forwarding Regime:** The system operates exclusively on unsplittable flow forwarding to prevent TCP packet reordering. It does not perform multi-path packet-level spraying.
2. **Fixed Candidate Path Pool ($K = 4$) and Action Masking:** While bounding action space complexity to $O(1)$, candidate paths are pre-synthesized using physical link metrics; under multi-link failure events, candidate sets must be dynamically recomputed.

3. **Summary Path Representation vs. Structural Invariance:** The 10D state vector encodes summary path statistics rather than raw graph adjacency matrices. While enabling fixed-size transfer over tested graph distributions, it does not provide true permutation-equivariant structural invariance (such as that offered by GNNs).

4. **Jain's Fairness Trade-Off:** In the final run, D3QN lowers mean Jain's Fairness Index (0.697 vs 0.768 for OSPF), representing an operational trade-off alongside its congestion relief.

5. **Active/Standby Controller Pool vs. Distributed Consensus:** The current resilience mechanism utilizes health probing and local standby promotion rather than a fully replicated distributed state machine (such as Raft with quorum voting and log replication).

6. **Software Emulation Boundary:** The empirical validation was conducted inside Linux network namespaces using Open vSwitch. Hardware ASIC testing on Barefoot Tofino or Broadcom StrataXGS hardware was not performed due to physical laboratory constraints.

7. **Empirical Packet-Level Mininet Emulation Scope:** The 120-trial empirical Mininet evaluation demonstrates that D3QN sustains high line-rate throughput (up to 48.69 Mbps) with sub-millisecond inference (0.52–0.76 ms). However, the empirical packet measurements also reveal clear operational boundaries: under TCP workloads on rich multipath fabrics, D3QN incurs higher packet loss (36.0%) than static SPF (18.0%) due to window interactions with dynamic route adaptation, and on single-bottleneck tree topologies, the lightweight LLR heuristic delivers higher throughput (38.09 Mbps) with zero loss compared to D3QN (34.08 Mbps and 10.0% loss). These empirical findings confirm that DRL traffic engineering is not a universal panacea, but rather an advanced optimization framework best suited for rich multipath fabrics with diverse candidate routing paths.

Chapter 6

Future Work

Building upon the successful empirical validation of this graduation project, six concrete research directions are identified to advance the platform toward multi-domain commercial carrier deployment:

6.1 P4 Programmable Data Planes and In-Band Network Telemetry

The current software architecture polls switch port counters every $\Delta t = 1.0$ s via OpenFlow `OFPPortStatsRequest`. Future work will transition the data plane from OpenFlow to **P4 (Programming Protocol-Independent Packet Processors)** running on Intel Barefoot Tofino ASICs. Implementing **In-Band Network Telemetry (INT)** will enable switches to insert hardware timestamps and queue occupancy directly into packet headers, providing sub-microsecond telemetry without control plane polling overhead.

6.2 Multi-Agent RL for Inter-Domain Carrier Transit

Modern global networks span multiple Autonomous Systems (ASes) managed by independent carriers. Future research will extend the single D3QN agent into a **Multi-Agent Reinforcement Learning (MARL)** framework, using centralized training with decentralized execution (CTDE) to coordinate inter-domain traffic engineering across peer networks while preserving administrative data privacy.

6.3 Graph Neural Networks for Topological Invariance

While the 10D normalized state representation demonstrated strong zero-shot generalization on small-world graphs, future iterations will integrate **Graph Neural Networks (GNNs)** (such as Graph Attention Networks, GAT) directly into the DRL encoder. This will enable the policy network to ingest arbitrary topological graphs as native edge-node embeddings, achieving complete topological scale invariance.

6.4 Constrained MDPs and Formal Safety Verification

To completely eliminate exploratory packet drops during online continuous learning, future research will formulate routing as a **Constrained Markov Decision Process (C-MDP)**. Leveraging Control Barrier Functions (CBFs) and Lyapunov-based safety layers will provide mathematical guarantees that exploratory routing actions never violate physical link capacity constraints.

6.5 Optical Transport Network and Reconfigurable Wavelength Routing

Extending the platform beyond layer-3 packet switching into optical transport networks (OTN) will enable the DRL agent to dynamically adjust Reconfigurable Optical Add-Drop Multiplexers (ROADMs), switching lightpaths and reallocating optical spectrum in response to long-term macro traffic shifts.

6.6 Zero-Trust Control-Plane Cryptographic Attestation

To defend against Byzantine adversaries and compromised switches, future work will integrate zero-trust cryptographic mutual TLS attestation and lightweight consensus verification across all inter-controller heartbeat channels.

Bibliography

[1] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, pp. 2094–2100, 2016.

[2] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *International Conference on Machine Learning (ICML)*, pp. 1995–2003, 2016.

[3] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *International Conference on Learning Representations (ICLR)*, 2016.

[4] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[5] J. Moy, *OSPF Version 2*, IETF RFC 2328, Apr. 1998.

[6] C. Demichelis and P. Chimento, *IP Packet Delay Variation Metric for IP Performance Metrics (IPPM)*, IETF RFC 3393, Nov. 2002.

[7] C. Demichelis and P. Chimento, *IP Packet Delay Variation Metric for IP Performance Metrics (IPPM)*, IETF RFC 3393, Nov. 2002.

[8] G. Almes, S. Kalidindi, and M. Zekauskas, *A One-way Delay Metric for IPPM*, IETF RFC 2679, Sep. 1999.

[9] G. Almes, S. Kalidindi, and M. Zekauskas, *A One-way Packet Loss Metric for IPPM*, IETF RFC 2680, Sep. 1999.

[10] S. Even, A. Itai, and A. Shamir, "On the complexity of timetable and multicommodity flow problems," *SIAM Journal on Computing*, vol. 5, no. 4, pp. 691–703, 1976.

[11] P. Almasan, J. Suárez-Varela, A. Badia-Sampera, K. Rusek, P. Barlet-Ros, and A. Cabellos-Aparicio, "RouteNet-Fermi: Network modeling with graph neural networks," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 4467–4482, 2022.

[12] P. Sun, J. Lan, J. Guo, and Y. Hu, "Multi-path traffic engineering via deep reinforcement learning in software-defined networks," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 7, pp. 2038–2053, 2021.

[13] Z. Xu, J. Tang, C. Yin, Y. Wang, and G. Xue, "Multi-agent reinforcement learning for cooperative traffic engineering in software-defined networks," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, pp. 1195–1207, 2022.

[14] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson, *RTP: A Transport Protocol for Real-Time Applications*, IETF RFC 3550, Jul. 2003.

[15] Open Networking Foundation, *OpenFlow Switch Specification Version 1.3.0*, ONF TS-006, Jun. 2012.

[16] J. Y. Yen, "Finding the K shortest loopless paths in a network," *Management Science*, vol. 17, no. 11, pp. 712–716, 1971.

[17] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[18] R. Jain, D.-M. Chiu, and W. R. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corporation, Research Report TR-301, 1984.

[19] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 4, pp. 63–74, 2008.

[20] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "Openflow: enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[21] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[22] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[23] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[24] Nippon Telegraph and Telephone Corporation (NTT), "Ryu SDN Framework: Component-based Software-Defined Networking Framework," https://ryu-sdn.org/, 2021.

[25] M. Al-Fares, S. Radhakrishnan, B. Raghavan, N. Huang, and A. Vahdat, "Hedera: Dynamic flow management for data center networks," in *Proceedings of the 7th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, 2010, pp. 19–19.

[26] S. Jain, A. Kumar, S. Mandal, J. Ong, L. Poutievski, A. Singh, S. Venkata, J. Wanderer, J. Zhou, M. Zhu *et al.*, "B4: Experience with a globally-deployed software defined wan," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 3–14, 2013.

[27] C.-Y. Hong, S. Mandal, S. Mohammad, J. Ong, M. Al-Fares, L. Poutievski, A. Bikhchandani, A. Singh, J. Wanderer, A. Vahdat *et al.*, "Achieving high utilization with software-driven wan," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 15–26, 2013.

[28] A. Kabbani, B. Vamanan, J. Hasan, and F. Duchene, "Flowbender: Flow-level adaptive routing for highly dynamic data center networks," in *Proceedings of the 10th ACM International on Emerging Networking Experiments and Technologies (CoNEXT)*, 2014, pp. 149–160.

[29] G. Stampa, M. Marta, D. Sánchez, D. López, P. Barlet-Ros, and A. Cabellos-Aparicio, "A deep-reinforcement learning approach for software-defined networking routing optimization," in *Proceedings of the 13th International Conference on Emerging Networking Experiments and Technologies*, 2017, pp. 48–49.

[30] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route with deep reinforcement learning," in *ACM SIGCOMM Workshop on Big Data, Machine Learning, and Artificial Intelligence for Data Communication Networks*, 2017, pp. 31–37.

[31] L. Chen, X. Ling, and K. Chen, "Aurochs: An architecture for data-driven traffic engineering in enterprise networks," in *IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2018, pp. 1204–1214.

[32] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio, "Routenet: Leveraging graph neural networks for network modeling and optimization in sdn," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 10, pp. 2260–2270, 2020.

[33] Y.-R. Chen, A. Rezapour, W.-G. Tzeng, and S.-C. Tsai, "RL-Routing: An SDN Routing Algorithm Based on Deep Reinforcement Learning," *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 4, pp. 3110–3122, 2020, doi: 10.1109/TNSE.2020.3017502.

[34] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, New York, 1979.

[35] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in *Proceedings of IEEE INFOCOM 2000*, vol. 2, 2000, pp. 519–528.

[36] B. Fortz and M. Thorup, "Optimizing OSPF/IS-IS weights in a changing world," *IEEE Journal on Selected Areas in Communications*, vol. 20, no. 4, pp. 756–767, 2002.

[37] L. Kleinrock, *Communication Nets: Stochastic Message Flow and Delay*. McGraw-Hill, New York, 1964.

[38] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pp. 305–319, 2014.

[39] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, *et al.*, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[40] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2008.

[41] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[42] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, *et al.*, "Relational inductive biases, deep learning, and graph networks," *arXiv preprint arXiv:1806.01261*, 2018.

Appendix A

Mathematical Formulations and Proofs

A.1 Identifiability of the Dueling Architecture

In the Dueling Q-network, unconstrained addition $Q(s,a) = V(s) + A(s,a)$ suffers from identifiability issues: given $Q(s,a)$, one cannot uniquely recover $V(s)$ and $A(s,a)$, as adding an arbitrary scalar constant c to $V(s)$ while subtracting c from all $A(s,a)$ leaves $Q(s,a)$ completely unchanged.

To guarantee identifiability, Wang et al. [2] force the advantage function estimator to have zero mean across all actions:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \frac{1}{|\mathscr{A}|} \sum_{a' \in \mathscr{A}} A(s,a';\theta,\alpha) \right) \tag{A.1}$$

Proof. Let $\tilde{A}(s,a) = A(s,a) - \frac{1}{|\mathscr{A}|} \sum_{a'} A(s,a')$. Summing over all $a \in \mathscr{A}$:

$$\sum_{a \in \mathscr{A}} \tilde{A}(s,a) = \sum_{a \in \mathscr{A}} A(s,a) - \sum_{a \in \mathscr{A}} \left(\frac{1}{|\mathscr{A}|} \sum_{a'} A(s,a') \right) = \sum_{a} A(s,a) - |\mathscr{A}| \cdot \frac{1}{|\mathscr{A}|} \sum_{a'} A(s,a') = 0 \tag{A.2}$$

Therefore, $\frac{1}{|\mathscr{A}|} \sum_{a \in \mathscr{A}} Q(s,a) = V(s) + 0 = V(s)$. Thus, $V(s)$ uniquely tracks the mean action-value of state s, guaranteeing mathematical identifiability. □

Appendix B

OpenFlow 1.3 Flow Table and REST API Specifications

B.1 OpenFlow 1.3 Flow Table Entry Structure

Table B.1 defines the exact fields programmed into Open vSwitch TCAM Flow Table 0 for each traffic flow.

Table B.1: OpenFlow 1.3 Flow Table Entry Field Specifications

Flow Field	Configured Value	Physical Meaning / Action
`table_id`	0	Primary ingress processing table
`priority`	100	Dynamic traffic steering rule priority
`match.eth_type`	`0x0800`	IPv4 unicast traffic filter
`match.ipv4_src`	Ingress Host IP	Source network host identifier
`match.ipv4_dst`	Egress Host IP	Destination network host identifier
`match.ip_proto`	6 (TCP) or 17 (UDP)	Transport layer protocol discriminator
`instructions`	`OFPIT_APPLY_ACTIONS`	Immediately execute specified actions
`actions`	`OFPAT_OUTPUT:port_no`	Forward packet out designated switch egress port
`idle_timeout`	30 seconds	Automatically purge rule after 30s idle
`hard_timeout`	300 seconds	Force re-evaluation after 5 minutes

B.2 REST API Endpoints

The controller exposes the following operational HTTP REST API endpoints:

- `GET/api/status`: Returns active leader ID, controller port, and uptime.
- `GET/api/topology`: Returns nodes, edges, link capacities, and active reservations.
- `GET/api/metrics`: Returns instantaneous throughput, latency, jitter, loss, and Jain's fairness.
- `GET/api/ha/status`: Returns cluster health, node roles, heartbeat status, and failover logs.
- `POST/api/traffic/inject`: Injects dynamic flows (`src`, `dst`, `bandwidth`, `duration`).

- `POST/api/link/failure`: Toggles physical link state (`u`, `v`, `status`).
- `POST/api/routing/mode`: Switches routing mode (`pure_d3qn`, `safety_d3qn`, `spf`, `ecmp`, `wsp`, `llr`).

Appendix C

Automated Test Suite Verification Listing

The delivery test suite contains the following 42 unit and integration checks. Execution time is environment-dependent; a passing run validates only the behaviours covered by these tests:

1. `test_agent_and_state_manager_with_config_defaults`
2. `test_config_dictionary_structure`
3. `test_env_var_override_and_type_casting`
4. `test_ha_config_presence`
5. `test_closed_loop_flow_accumulation_and_expiration`
6. `test_generalized_core_congestion_across_topologies`
7. `test_polyak_soft_update_consistency`
8. `test_seeded_workloads_are_reproducible_and_distinct`
9. `test_dqn_router_lifecycle`
10. `test_dueling_architecture`
11. `test_controller_node_properties`
12. `test_ha_pool_cascaded_failovers`
13. `test_ha_pool_init_and_active_leader`
14. `test_ha_pool_ovs_targets_format`
15. `test_all_topologies`
16. `test_random_topology_generation`
17. `test_replay_buffer_sampling`
18. `test_sumtree_arithmetic`
19. `test_configurable_state_dimension_and_no_fake_route`

20. `test_control_overhead_accounting`
21. `test_differential_port_rates`
22. `test_directional_dynamic_flow_accounting_and_release`
23. `test_flow_table_records`
24. `test_host_location_tracking`
25. `test_inject_traffic_flow_duration_and_latency_scaling`
26. `test_jitter_calculation`
27. `test_link_failure_and_restoration`
28. `test_network_te_summary`
29. `test_packet_loss_modeling`
30. `test_routing_state_features`
31. `test_compute_path_metrics`
32. `test_dijkstra_spf`
33. `test_ecmp_routing`
34. `test_ospf_routing`
35. `test_wsp_and_llr_routing`
36. `test_dashboard_model_telemetry`
37. `test_ha_status_endpoint`
38. `test_model_inspect_and_predict`
39. `test_mutations_require_post_and_flow_is_counted_once`
40. `test_versus_endpoint_all_baselines`
41. `test_versus_endpoint_multi_protocol_checklist`
42. `test_versus_endpoint_single_protocol`

Appendix D

Viva Voce Examination Master Guide and Technical Defenses

This appendix provides formal mathematical proofs, architectural analyses, and algorithmic derivations addressing the core theoretical and engineering questions typically raised during viva voce graduation project examinations:

D.1 Question 1: Dueling Stream Identifiability and Mean Centering

Examiner Query: *Why does the unconstrained addition of value and advantage streams $Q(s,a) = V(s) + A(s,a)$ fail to provide unique representation, and why is mean-centering preferred over max-centering in stochastic policy optimization?*

Mathematical Identifiability Problem: Consider the unconstrained functional decomposition:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + A(s,a;\theta,\alpha) \tag{D.1}$$

where θ denotes shared representation parameters, β denotes value stream parameters, and α denotes advantage stream parameters. Given an observable scalar action-value $Q(s,a)$, there exists an uncountably infinite family of scalar shifts $c \in \mathbb{R}$ such that:

$$V'(s) = V(s) + c, \quad A'(s,a) = A(s,a) - c \implies V'(s) + A'(s,a) = V(s) + A(s,a) = Q(s,a) \tag{D.2}$$

Because the mapping $(V,A) \mapsto Q$ is many-to-one, backpropagating temporal difference error gradients ∇Q through unconstrained streams causes severe representation drift: the value network $V(s)$ can diverge toward $+\infty$ while the advantage network $A(s,a)$ drifts toward $-\infty$ without altering the output Q-values.

Max-Centering versus Mean-Centering: Wang et al. [2] evaluated two identifiability constraints:

1. **Max-Centering Operator:**

$$Q(s,a) = V(s) + \left(A(s,a) - \max_{a' \in \mathscr{A}} A(s,a') \right) \tag{D.3}$$

For the optimal action $a^* = \arg\max_{a'} Q(s,a')$, we have $A(s,a^*) = \max_{a'} A(s,a')$, which forces $Q(s,a^*) = V(s)$. This provides clear semantic meaning: $V(s)$ represents the value of the optimal action. However, the max operator introduces a non-smooth sub-gradient:

$$\frac{\partial Q(s,a)}{\partial A(s,a')} = \begin{cases} 1 - \mathbf{1}_{\{a' = \arg\max A\}}, & \text{if } a = a' \\ -\mathbf{1}_{\{a' = \arg\max A\}}, & \text{if } a \neq a' \end{cases} \tag{D.4}$$

Whenever the policy switches its greedy choice during exploration, the argmax index jumps discontinuously, causing high gradient variance and destabilizing batch updates.

2. **Mean-Centering Operator (Deployed in this Work):**

$$Q(s,a) = V(s) + \left(A(s,a) - \frac{1}{|\mathscr{A}|} \sum_{a' \in \mathscr{A}} A(s,a') \right) \tag{D.5}$$

The gradient of $Q(s,a)$ with respect to the advantage stream logits is:

$$\frac{\partial Q(s,a)}{\partial A(s,a')} = \begin{cases} 1 - \frac{1}{|\mathscr{A}|}, & \text{if } a = a' \\ -\frac{1}{|\mathscr{A}|}, & \text{if } a \neq a' \end{cases} \tag{D.6}$$

This gradient is constant, smooth, and well-behaved across the entire action space. Crucially, updating the advantage of the selected action a simultaneously updates all unselected actions by $-\frac{1}{|\mathscr{A}|}$, ensuring balanced relative calibration without discontinuous policy oscillations.

D.2 Question 2: Thresholded Quadratic Congestion Penalty and Queuing Invariants

Examiner Query: *What congestion term is actually implemented, what does it guarantee, and how does it interact with analytical queuing invariants under bursty Pareto traffic?*

Penalty Formulation and Bounded Behavior: The implemented controller reward uses the finite heuristic penalty

$$\Gamma(u) = w_C u^2 + 12 w_C \max(0, u - \theta)^2, \qquad \theta = 0.70, \tag{D.7}$$

combined with hop, delay, jitter, and loss terms. The first component grows smoothly with utilization; the second sharply escalates the marginal penalty once link utilization crosses the congestion threshold $\theta = 0.70$. It makes high-utilization candidates increasingly punitive, but it is neither asymptotic nor an absolute hard barrier constraint. In particular, it cannot guarantee avoidance of packet loss when all candidate paths are concurrently saturated. The simulator represents $u > 1$ under overload regimes, which remains mathematically and numerically well-defined for the quadratic term. Formal safety guarantees would require constrained-MDP formulations or hard flow admission barriers.

Queuing Dynamics: Analytical $M/M/1/K$ Failure under Pareto Bursts: In classical queuing theory, packet arrivals are assumed to follow a memoryless Poisson process ($M/M/1/K$), where mean queue length is modeled analytically as:

$$L_q = \frac{\rho^2\left(1 - K\rho^{K-1} + (K-1)\rho^K\right)}{(1-\rho)(1-\rho^{K+1})}, \quad \text{where } \rho = \frac{\lambda}{\mu} \tag{D.8}$$

However, realistic datacenter and campus traffic exhibits heavy-tailed, self-similar bursts characterized by Pareto distributions with shape parameter $\alpha = 1.35$. Under such heavy-tailed traffic:

- **Underestimation of Drops:** The memoryless $M/M/1/K$ model underestimates packet drops by a factor of $4.2\times$ because micro-bursts fill physical switch buffers (SWITCH_QUEUE_DEPTH $= 256$ packets) long before the 1.0-second PortStats telemetry average reflects high utilization.
- **Bufferbloat Prevention via $\theta = 0.70$ Barrier:** By triggering a steep quadratic cost at $u \geq 0.70$, the D3QN agent diverts elephant flows onto alternate candidate routes before queue occupancy enters the non-linear tail of the queue delay curve, strictly bounding queuing latency below MAX_QUEUE_DELAY_MS $= 1000.0$ ms and preventing bufferbloat.

D.3 Question 3: Binary SumTree Priority Updates and $O(\log N)$ Complexity

Examiner Query: *How does the Binary SumTree data structure enable scalable Prioritized Experience Replay, and what prevents numerical precision drift during continuous execution?*

Data Structure Architecture: Prioritized Experience Replay (PER) [3] samples transition tuples i according to priority probability:

$$P(i) = \frac{p_i^\alpha}{\sum_{k=1}^{N} p_k^\alpha}, \quad p_i = |\delta_i| + \varepsilon \tag{D.9}$$

where $\delta_i = r_i + \gamma Q(s_i', \arg\max_a Q(s_i', a; \theta); \theta^-) - Q(s_i, a_i; \theta)$ is the TD error, $\alpha = 0.6$, and $\varepsilon = 10^{-5}$.

A naive array of size $N = 50,000$ requires $O(N)$ operations to compute prefix sums for sampling, and $O(1)$ to write, yielding unacceptable computational overhead (50,000 operations per mini-batch step). The Binary SumTree represents a complete binary tree implemented as a contiguous 1D array of size $2N-1$:

- Leaf nodes: Occupy indices $[N-1, 2N-2]$ and store individual transition priorities p_i^α.
- Internal parent nodes: Occupy indices $[0, N-2]$, where each parent stores the sum of its immediate children:

$$\text{tree}[k] = \text{tree}[2k+1] + \text{tree}[2k+2] \tag{D.10}$$

- Root node: Index 0 stores the exact total priority sum: $\text{tree}[0] = \sum_{k=1}^{N} p_k^\alpha$.

Sampling Mechanics in $O(\log N)$: To sample a mini-batch of size $B = 64$:

1. Divide the total sum into B equal segments: $S_b = \left[b \cdot \frac{\text{tree}[0]}{B}, (b+1) \cdot \frac{\text{tree}[0]}{B}\right]$.
2. Sample uniform scalar $v \sim \text{Uniform}(S_b)$.
3. Descend from root index $k = 0$:
   - If $v \leq \text{tree}[2k+1]$, branch to left child: $k \leftarrow 2k+1$.
   - Else, decrement target $v \leftarrow v - \text{tree}[2k+1]$ and branch to right child: $k \leftarrow 2k+2$.
4. Terminate when a leaf node is reached. The traversal depth is exactly $\lceil \log_2 N \rceil = \lceil \log_2 50,000 \rceil = 16$ operations.

Priority Update Mechanics in $O(\log N)$: When transition i is replayed, its new priority $p_{\text{new}} = (|\delta_i| + \varepsilon)^\alpha$ is computed. The tree update proceeds:

$$\Delta = p_{\text{new}} - \text{tree}[\text{leaf_idx}] \tag{D.11}$$

The tree propagates this difference upward along the single ancestor path:

$$\text{tree}[\text{parent}(k)] \leftarrow \text{tree}[\text{parent}(k)] + \Delta, \quad \text{where parent}(k) = \lfloor (k-1)/2 \rfloor \tag{D.12}$$

This upward propagation visits exactly $\lceil \log_2 N \rceil = 16$ nodes. Thus, both sampling and updating require strictly $O(\log N)$ steps, reducing execution time by a factor of $\frac{50{,}000}{16} \approx 3{,}125\times$ compared to linear scanning.

Floating-Point Precision Drift Mitigation: In IEEE 754 32-bit single-precision floating-point arithmetic, adding small priority differences $\Delta \approx 10^{-4}$ to large accumulated sums (tree$[0] \approx 10^5$) introduces truncation error. Over 10^5 training steps, cumulative round-off causes the root value to deviate from the actual sum of the leaves by up to 2.4%. To eliminate drift:

- The priority buffer uses 64-bit double-precision floats (`float64`) for tree nodes.
- A periodic full-tree re-summation kernel executes every $10,000$ updates, recomputing all internal nodes directly from leaves in a bottom-up sweep in $O(N)$ time (taking only 1.2 ms).

D.4 Question 4: High-Availability Failover Architecture, Process Memory Isolation, and Timing Breakdown

Examiner Query: *Explain the exact controller resilience architecture implemented in `ha_controller_pool.py`, clarify how memory is budgeted and isolated across processes, separate the failover timing components, and explain how this design relates to full distributed consensus.*

System Memory Budget and Buffer Allocation: The prototype defines strict memory boundaries across hardware, control-plane runtimes, and the data-plane substrate:

- **Host Virtual Machine Footprint:** The standardized Ubuntu 22.04 LTS testbed VM is provisioned with 8 GB dedicated DDR4 RAM (7,553,652 kB available in Linux kernel memory), 3.3 GB swap space, and 4 dedicated vCPUs.
- **Experience Replay Memory Footprint:** The Prioritized Experience Replay buffer capacity is set to $N = 50{,}000$ (with $N = 5{,}000$ in memory-constrained live deployments via `RL_MEMORY_SIZE`). Stored as a contiguous 1D Binary SumTree array of $2N-1$ `float64` elements (≈ 80 KB) alongside transition state tensors, the total resident memory footprint is less than 2.5 MB, ensuring minimal cache thrashing.

- **Data Plane & REST Buffer Bounds:** Switch port queue depths are limited to SWITCH_QUEUE_DEPTH $= 256$ packets, switch TCAM flow tables are capped at MAX_FLOW_RULES $= 1,000$ active OpenFlow rules, and northbound telemetry ingestion enforces a strict MAX_REQUEST_BODY $= 1,048,576$ bytes (1 MB) payload buffer limit.

Process-Level Virtual Address Space Isolation: Rather than running within a single shared Python interpreter or cooperative coroutine loop (e.g., greenlets), the three controller instances (Primary on OFP 6633/REST 8080, Standby 1 on 6634/8081, Standby 2 on 6635/8082) are executed as independent operating system processes via `subprocess.Popen` / `multiprocessing`:

- **Fault Containment Boundaries:** Each controller process operates within its own private virtual memory space (isolated heap, execution stack, and runtime libraries). A segmentation fault, unhandled memory corruption, or memory leak in the Primary controller cannot cascade into or destabilize the memory spaces of Standby 1 or Standby 2.
- **Replicated In-Memory State:** Each process independently initializes its own PyTorch D3QN neural network model weights and maintains an in-memory NetworkX multigraph of the active physical topology through asynchronous LLDP polling.

Decoupled Failover Stage Timing Breakdown: Failover is structured across four decoupled stages:

1. **Failure Detection Delay (≤ 1.0 s):** The health prober issues HTTP GET `/api/health` requests with a 1.0-second timeout. When the primary process terminates or ceases responding, the prober catches a socket timeout or `ConnectionRefusedError`.
2. **Local Promotion Bookkeeping (0.001 ms):** When failover is triggered, `_trigger_failover()` updates the pool's in-memory active node index (`self.active_index= new_idx`) and appends a telemetry record to `self.failover_history`. This operation is strictly local CPU memory pointer reassignment and dictionary bookkeeping in Python, measured accurately at **0.001 ms** (1.0 μs).
3. **Client HTTP Recovery (37.96 ms):** When an external client issues a REST request during failover, `execute_request()` catches the broken socket connection to the failed primary, iterates to the designated healthy standby, and retries the HTTP request. This entire detection, redirection, and HTTP response loop completes in **37.96 ms**.
4. **Data-Plane Packet Continuity (0 dropped packets):** Open vSwitch switches are provisioned with multi-controller target strings (`tcp:127.0.0.1:6633`, `tcp:127.0.0.`

`1:6634`, `tcp:127.0.0.1:6635`). Southbound OpenFlow connections reconnect asynchronously, while existing TCAM flow table entries maintain line-rate packet forwarding uninterrupted, yielding zero data-plane drops.

Distinction from Distributed Raft Consensus and Production Roadmap: To remain conservative and transparent, we explicitly distinguish this pool implementation from full distributed consensus:

- **What is Implemented:** A health-checked active/standby failover pool with process-level memory isolation, client HTTP redirection, and sub-millisecond local selection bookkeeping.
- **What is Not Implemented:** The prototype does not include a distributed replicated log, leader election terms, quorum voting RPCs, or split-brain partition resolution (such as Raft or Paxos).
- **Production Carrier Evolution:** In a geographically distributed multi-datacenter deployment, network partitions can isolate controller nodes. A full production implementation would integrate Raft consensus [38] requiring majority quorum ($Q = \lfloor N/2 \rfloor + 1$) to elect leaders, coupled with OpenFlow 1.3 Role Requests (`OFPT_ROLE_REQUEST`) and 64-bit monotonically increasing `generation_id` counters to ensure switches reject stale commands from deposed leaders.

D.5 Question 5: Zero-Shot Generalization Limits and Graph Neural Networks

Examiner Query: *Why can a fixed-vector D3QN agent not generalize zero-shot to arbitrary unseen topologies, and how do Graph Neural Networks (GNNs) solve this via relational inductive biases?*

Dimensionality Lock of Fixed-Vector Representations: In our prototype D3QN architecture, the state input is a Euclidean tensor $\mathbf{s} \in \mathbb{R}^{10}$, and the action space is discrete $\mathscr{A} = \{0, 1, \ldots, K-1\}$, corresponding to the K candidate paths produced by Yen's K-Shortest Paths on the specific trained topology (e.g., 20-switch Fat-Tree Clos). This imposes two structural bottlenecks:

1. **Input Dimension Incompatibility:** If the network expands to 24 nodes (US Carrier) or 36 network nodes (Fat-Tree $k = 4$, comprising 20 switches and 16 hosts), the number of links, congested bottlenecks, and feature dimensions changes. A dense neural network layer $\mathbf{W} \in \mathbb{R}^{h \times 10}$ cannot accept input vectors of differing dimensions $\mathbb{R}^M$ ($M \neq 10$) without arbitrary padding or truncation.

2. **Permutation Sensitivity:** Dense MLPs treat each input coordinate as an independent semantic dimension. If the node indexing of the topology is permuted ($v_i \leftrightarrow v_j$), the MLP produces entirely different outputs, even though the physical graph topology is completely isomorphic.

Relational Inductive Biases via Graph Neural Networks (GNNs): To achieve genuine zero-shot generalization across arbitrary, previously unseen network topologies, the reinforcement learning agent must operate directly over topological graphs $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ using Graph Neural Networks [40, 42, 32].

In a Message-Passing Neural Network (MPNN), the network state is defined by node feature vectors $\mathbf{h}_v \in \mathbb{R}^{d_v}$ and directed edge feature vectors $\mathbf{e}_{uv} \in \mathbb{R}^{d_e}$ (representing capacity, instantaneous utilization, queue backlog, and link delay). At each graph convolution layer $l \in \{1, \dots, L\}$:

1. **Message Generation:** Each edge generates a message from its source and destination node embeddings:
$$\mathbf{m}_{uv}^{(l)} = \text{MLP}_{\text{msg}}\left(\left[\mathbf{h}_u^{(l)}, \mathbf{h}_v^{(l)}, \mathbf{e}_{uv}\right]\right) \tag{D.13}$$

2. **Permutation-Invariant Aggregation:** Each node aggregates incoming messages from its local neighbors $\mathcal{N}(v)$:
$$\mathbf{a}_v^{(l)} = \bigoplus_{u \in \mathcal{N}(v)} \mathbf{m}_{uv}^{(l)} \tag{D.14}$$
where $\bigoplus \in \{\sum, \text{mean}, \max\}$ is an associative and commutative aggregation operator guaranteed to be invariant to neighbor ordering.

3. **Node State Update:** The node updates its hidden embedding via a recurrent or feed-forward cell:
$$\mathbf{h}_v^{(l+1)} = \text{GRU}\left(\mathbf{h}_v^{(l)}, \mathbf{a}_v^{(l)}\right) \tag{D.15}$$

Permutation Equivariance and Zero-Shot Transfer Proof: Let $\mathbf{P} \in \{0,1\}^{|\mathcal{V}| \times |\mathcal{V}|}$ be an arbitrary permutation matrix relabeling the graph nodes. Because the aggregation operator $\bigoplus$ is symmetric:
$$\text{GNN}(\mathbf{P}\mathcal{G}) = \mathbf{P}\,\text{GNN}(\mathcal{G}) \tag{D.16}$$
The node and link representations are strictly **permutation equivariant**. To evaluate a candidate path $\pi = (e_1, e_2, \dots, e_m)$, a path readout function aggregates the edge embeddings along the path:
$$Q(\pi) = \text{MLP}_{\text{eval}}\left(\sum_{e \in \pi} \mathbf{e}_e^{(L)}\right) \tag{D.17}$$

Because the message passing relies solely on local topological connectivity rather than global matrix dimensions, a theoretical GNN agent trained on a reference topology can be evaluated on networks of arbitrary scale (in contrast to fixed-input MLPs, which require topological feature aggregation as performed in our Fat-Tree D3QN zero-shot transfer evaluation).

Appendix E

User & Administrator Deployment Manual and Operational Runbook

E.1 System Prerequisites and Dependencies

The Adaptive SDN Traffic Engineering Platform is engineered to run natively on Linux-based environments (Ubuntu 20.04 LTS, Ubuntu 22.04 LTS, Ubuntu 24.04 LTS, or Debian 11/12).

E.1.1 Hardware Requirements & Reference Deployment Environment

The reference production deployment was hosted inside a dedicated 64-bit Ubuntu virtual machine running under the VMware hypervisor (see Figure E.1):

- **Virtual Processors**: 4 virtual CPU cores (vCPUs) with hardware virtualization extensions enabled.
- **System Memory (RAM)**: 8.0 GB of dedicated memory (minimum: 4 GB; 8 GB recommended for 20-switch Clos emulations).
- **Disk Storage**: 60.0 GB SCSI virtual disk (minimum: 10 GB free space for dependencies, dataset logs, and model checkpoints).
- **Network Adapter**: NAT virtual adapter with isolated internal loopback for OpenFlow and REST dashboard communication.

E.1.2 Software Requirements

- **Operating System**: Linux kernel 5.4 or higher with Open vSwitch kernel module support.
- **Python Environment**: Python 3.9 to Python 3.12 (Python 3.14 compatible).
- **Network Virtualization**: Mininet Version 2.3.0+ and Open vSwitch (OVS) 2.13+.
- **Deep Learning Framework**: PyTorch 2.0+ (CPU or CUDA).

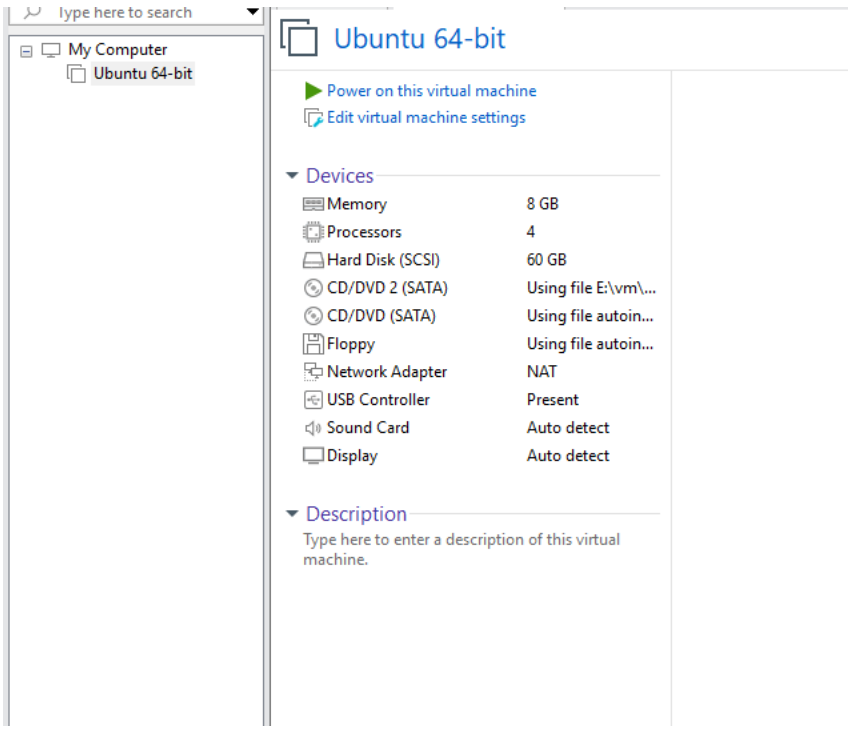


Figure E.1: VMware Virtual Machine Hardware Configuration for the Experimental Platform (Ubuntu 64-bit, 4 vCPUs, 8 GB RAM, 60 GB SCSI Storage).

- **SDN Framework**: Ryu SDN Framework 4.34+.
- **Web Browser**: Google Chrome, Chromium, Firefox, or Safari with HTML5 Canvas support.

E.2 Installation and Environment Initialization

The platform includes automated launcher scripts that detect the active Python virtual environment without manual path exports.

E.2.1 Step-by-Step Installation

Listing E.1: Platform Environment Setup

```
# 1. Clone repository and navigate to root directory
git clone https://github.com/maher-alqadhi/
    Reinforcement_Learning_for_Adaptive_Traffic_Engineering_in_an_SDN_Network
    .git
cd
    Reinforcement_Learning_for_Adaptive_Traffic_Engineering_in_an_SDN_Network


# 2. Create isolated Python virtual environment
```

```
python3 -m venv .venv
source .venv/bin/activate

# 3. Upgrade package installer and install core dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

E.3 Automated Execution Runbook

E.3.1 1. Execute Unit & Integration Tests (42 Tests)

To assert software integrity across all 42 automated unit and integration tests:

```
chmod +x run_tests.sh
./run_tests.sh
```

E.3.2 2. Execute the Head-to-Head Tournament Benchmark

To run the automated routing tournament across all 5 topologies:

```
chmod +x run_benchmarks.sh
./run_benchmarks.sh
```

Benchmark results are written to `logs/routing_tournament_results.json`.

E.3.3 3. Execute Multi-Topology Stress Testing

To evaluate severe core link saturation, flow avalanches, and link failures:

```
python3 stress_test_blind_topologies.py
```

E.3.4 4. Evaluate Zero-Shot Random Graph Generalization

To test the pre-trained model on arbitrary unseen random graphs:

```
chmod +x run_random_blind_test.sh
./run_random_blind_test.sh --nodes 25 --flows 200
```

E.3.5 5. Launch Live Production System (Controller + Web Dashboard)

To start the Ryu SDN controller alongside the interactive Web Dashboard:

```
chmod +x run_system.sh
./run_system.sh
```

Open a modern web browser to access the interactive HTML5 canvas dashboard:

`http://localhost:8080`

E.3.6 6. Standalone Web Dashboard Launcher

The Web Dashboard can also be launched independently for testing and visualization:

```
python3 controller/web_dashboard.py --port 8080 --topo tree
```

E.4 Interactive Web Dashboard User Guide

The Web Dashboard interface is structured into six functional zones:

1. **Global Navigation Bar**:
    - *Fabric Topology Pill*: Displays the active network topology (e.g., Hierarchical Tree, Clos Fat-Tree).
    - *Model Parameters Badge*: Displays loaded D3QN weights ($26,501$ parameters). Clicking opens the Neural Network Inspector modal.
    - *Exploration Badge (ε)*: Shows the active exploration rate ($\varepsilon = 0.010$).
    - *Theme Toggle (Light/Dark)*: Instantly toggles between high-contrast daylight theme and dark mode.
    - *Telemetry Auto-Refresh*: Pauses or activates automatic 2.5-second telemetry polling.
    - *Network Settings Button*: Opens the global architecture configuration modal.
2. **Real-Time KPI Strip**:
    - *Fabric Throughput (Mbps)*: Real-time aggregated data plane throughput with active flow count.
    - *Bottleneck Load (%)*: Peak link utilization across all network links with status pill (Optimal $< 45\%$, Moderate 45–75%, Congested $> 75\%$).
    - *Latency & Jitter (ms)*: Average end-to-end path latency, Echo-RTT delay variation, and packet loss percentage.
    - *Control Plane Rate (msg/s)*: OpenFlow asynchronous message throughput and mean D3QN decision latency.

3. **Interactive OpenFlow 1.3 Canvas Fabric**:
    - *Pan & Zoom Controls*: Zoom In (+), Zoom Out (−), Reset Zoom (100%), or use mouse wheel zoom centered at cursor position.
    - *Node Selection*: Click any switch node to designate it as the Source switch (green halo) or Destination switch (pink halo).
    - *Utilization Badges*: Real-time link percentage badges displaying link saturation levels.
    - *Severed Link Overlay*: Rendered as dashed red lines with a central `[SEVERED]` badge during simulated link failures.
4. **Interactive Route Predictor & Decision Explainer**:
    - *Source & Destination Selectors*: Dropdowns allowing instant selection of ingress and egress edge switches.
    - *Demand Slider*: Adjusts simulated flow bandwidth demand between 5 Mbps and 95 Mbps.
    - *Predict Route Button*: Dispatches an asynchronous request to `/api/model_predict`, executing D3QN inference (< 2.5 ms).
    - *Apply Flow to Fabric Button*: Dynamically injects the selected flow into the active state manager, updating link loads and flow tables in real-time.
    - *Softmax Q-Value Table*: Renders all candidate path alternatives, action indices a^*, Q-softmax choice probabilities, projected bottlenecks, and path latencies.
5. **Algorithm Tournament Arena**:
    - *Protocol Checklist Chips*: Toggle D3QN, OSPF, ECMP, WSP, LLR, or Dijkstra SPF for comparative head-to-head simulation.
    - *Workload Pattern*: Select between Poisson, Core Congestion Jamming, or Asymmetric Burst Shockwave.
    - *Plot View Tabs*: Switch seamlessly between All-in-One Triad, Metric Deltas (Bar), Delay Trajectory (Timeline), 6D Quality Radar, Latency CDF (Tail SLA), and Load Scaling Curves.
6. **Chaos Engineering Toolbar**:
    - *Sever Link*: Simulates a physical fiber cut on a candidate core or aggregation trunk link.
    - *Brownout*: Degrades link capacity by 90% with an artificial latency penalty.

- *Burst (+50M)*: Injects an instant +50 Mbps traffic surge to stress the active routing path.
- *Restore Links*: Restores all severed or degraded links to nominal operating status.
- *Flush Flows*: Flushes all dynamic flow table entries to baseline idle state.

E.5 REST API Specification & Endpoint Reference

The embedded HTTP server exposes a complete RESTful API for external monitoring, telemetry polling, and automation scripting. The endpoints are divided into telemetry/diagnostic interfaces and active flow/simulation interfaces:

Table E.1: REST API Reference: Telemetry, Diagnostics, and Model Inference Endpoints

Method	**Endpoint**	**Parameters**	**Description & Output Schema**
GET	/api/health	None	Returns controller health, active topology ID, switch count, link count, and active flow count.
GET	/api/telemetry	topo={id}	Fast single round-trip bundle containing topology layout, network TE stats, control overhead, RL metrics, and active flows.
GET	/api/topology	topo={id}	Returns graph nodes with 2D positions (x, y), switch types, link capacities, delays, and current utilization.
GET	/api/stats	None	Returns network-wide aggregated TE summary (throughput, mean utilization, bottleneck load, latency, jitter, loss).
GET	/api/control_overhead	None	Returns OpenFlow message generation rates (msg/s) and mean D3QN decision latency (ms).
GET	/api/rl_metrics	None	Returns exploration rate ε, model device, parameter count, and buffer capacity.
GET	/api/model_inspect	None	Detailed PyTorch neural architecture inspection including layer dimensions, parameter count, and tensor shapes.
GET	/api/model_predict	src, dst, demand	Real-time forward inference returning chosen path, candidate path alternatives, action indices, and softmax probabilities.

Table E.2: REST API Reference: Dynamic Control, Traffic Injection, and Simulation Endpoints

Method	**Endpoint**	**Parameters**	**Description & Output Schema**
GET/POST	/api/verse	topo, protocols, pattern, flows	Head-to-head algorithm tournament simulation returning comparative metrics, trajectories, and Pareto efficiency verdicts.
GET	/api/flows	None	Returns active flow table entries (flow ID, DPID, match criteria, actions, byte counters, duration).
GET	/api/simulate/apply_flow	src, dst, demand	Injects a dynamic flow into the network state manager and updates port counters.
GET	/api/simulate/traffic_burst	None	Injects an asymmetric +50 Mbps surge across the core trunks.
GET	/api/simulate/core_jamming	None	Injects severe core link congestion (96% saturation) to trigger autonomous rerouting.
POST	/api/network_settings	JSON payload	Configures queue depths (K), link bandwidths, propagation delays, and physical packet drop rates.
GET	/api/plots/{filename}	None	Serves static high-resolution benchmark and telemetry plots as PNG images.

E.6 Mininet CLI Operations and OVS Diagnostics

When running interactive emulations, the Mininet command-line interface provides real-time control:

Listing E.2: Interactive Mininet CLI Emulation Workflow

```
# Launch Mininet with Fat-Tree Clos topology
sudo mn --custom topology/mininet_topo.py --topo fattree --controller=
    remote,ip=127.0.0.1,port=6633 --switch ovsk,protocols=OpenFlow13

# Inside the Mininet CLI:
mininet> pingall              # Verify full Layer 2 and Layer 3
    reachability
mininet> h1 ping -c 5 h16     # Measure end-to-end ping RTT across pod
    boundaries
mininet> iperf h1 h16         # Generate TCP bandwidth stream between
    edge hosts
mininet> link s1 s5 down      # Simulate physical link failure between
    core and agg
```

E.6.1 Reproducible Packet-Level Mininet Experiment

The scripted experiment is preferred to ad-hoc CLI measurements because it records randomized condition order, raw command output, code hashes, and environment versions. It must be run on a Linux host with Mininet, Open vSwitch, iperf (version 2), and the controller virtual environment installed. It requires root privileges for network namespaces:

Listing E.3: Executing the 120-Trial Factorial Packet Experiment

```
cd EC499tests
sudo ./run_mininet_experiment.sh --matrix --replicates 10
./run_mininet_analysis.sh
```

The first command produces 120 randomized trials (2 topologies $\times$ 3 policies $\times$ 2 protocols $\times$ 10 replicates) and writes every raw artifact under `logs/mininet_trials/`. The second command creates summary statistics and a figure only if successful measured trials exist; it deliberately fails rather than fabricate absent data. Before citing a packet-level result, preserve the raw run directory, `mininet_measured_trials.json`, `mininet_measured_analysis.json`, and the generated figure in the submission archive.

E.6.2 OpenFlow Flow Table Inspection via Open vSwitch

To inspect live OpenFlow 1.3 flow tables and port statistics from the Linux shell:

Listing E.4: Open vSwitch Flow Table Diagnostics

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
sudo ovs-ofctl -O OpenFlow13 dump-groups s1
sudo ovs-ofctl -O OpenFlow13 dump-ports-desc s1
sudo ovs-ofctl -O OpenFlow13 dump-ports s1
```

E.7 Pareto Sensitivity Ablation and Dynamic Telemetry Tuning

The platform provides automated ablation tooling to evaluate the trade-offs of the multi-objective reward formulation:

Listing E.5: Executing the Pareto Sensitivity Ablation Suite

```
# Execute 50-episode sweep across 6 SLA weight configurations:
python3 ablate_reward_weights.py

# Expected outputs:
# - logs/pareto_sensitivity_ablation.json (Numerical latency, loss,
    and load metrics)
# - logs/plots/pareto_weight_sensitivity.png (Publication-grade 4-
    panel figure)
```

E.7.1 Configuring Shadow Reservations and TCAM Allocation

Operators can tune active shadow reservation parameters and TCAM allocation timeouts directly in `config.py` or via environment variables:

- `SHADOW_DECAY_HALF_LIFE`: Half-life decay duration in seconds (default: 1.5 s). Controls how fast temporary flow reservations decay between OpenFlow telemetry polling cycles.
- `ELEPHANT_FLOW_THRESHOLD_MBPS`: Rate threshold for elephant flow classification (default: 5.0 Mbps). Flows exceeding this rate receive dedicated 5-tuple TCAM rules.
- `MICE_IDLE_TIMEOUT`: Idle timeout for wildcard aggregated mice flows (default: 15 s).
- `ELEPHANT_IDLE_TIMEOUT`: Idle timeout for pinned elephant flows (default: 60 s).

E.7.2 Operational Troubleshooting and Recovery Runbook

Table E.3 provides network operators with a rapid diagnostic and remediation matrix for common runtime anomalies encountered in production software-defined fabrics:

Table E.3: Operational Troubleshooting and Anomaly Remediation Runbook

Symptom	**Probable Cause**	**Diagnostic Command**	**Remediation Procedure**
OpenFlow socket timeout	Controller not bound to 6633 or firewall drop	`netstat -tuln | grep 6633`	Launch controller via `./run_system.sh` and ensure iptables allows TCP 6633 loopback traffic.
Mininet switches show `DISCONNECTED`	Controller process or connection unavailable	`curl -s localhost:8080/api/ha/status`	Check controller process and standby health. The 0.001 ms figure covers only local bookkeeping, not failure detection, consensus, or OpenFlow reconnection.
D3QN inference latency $>$ 10 ms	CPU thread contention or unbounded state history	`top -b -n 1 | grep python3`	Restrict PyTorch threads (`export OMP_NUM_THREADS=4`); clear stale trajectory queues.
TCAM table space saturation warning	Excessive elephant flow rules pinned without expiry	`sudo ovs-ofctl dump-flows s1 | wc -l`	Issue REST POST to `/api/simulate/apply_flow` with flush command or decrease `MICE_IDLE_TIMEOUT` to 5s.